

Chapter 6: Database Design Using the E-R Model

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Outline

- Overview of the Design Process
- The Entity-Relationship Model
- Complex Attributes
- Mapping Cardinalities
- Primary Key
- Removing Redundant Attributes in Entity Sets
- Reducing ER Diagrams to Relational Schemas
- Extended E-R Features



Design Phases

- Initial phase -- Characterize fully the data needs of the prospective database users.
- Second phase -- Choosing a data model
- Final Phase -- Moving from an abstract data model to the implementation of the database

©Silberschatz, Korth and Sudarshan



Logical versus Physical

- Logical Design – Deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
- Physical Design – Deciding on the physical layout of the database



Logical Design

- Business decision – What attributes should we record in the database?
- Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas



Logical Design Alternatives

- Consider the information about the instructors.
  - Schema one:
    - Instructor = (ID, name, dept_name, salary)
  - Schema two:
    - In1 = (ID, name)
    - In2 = (ID, dept_name)
    - In3 = (ID, salary)
  - Schema three:
    - In1 = (ID, name)
    - In2 = (name, dept_name, salary)
- They contain the same set of attributes
- What are the differences?

©Silberschatz, Korth and Sudarshan



Design Pitfalls

- In designing a database schema, we must ensure that we avoid two major pitfalls:
  - Redundancy: a bad design may result in repeat information.
    - Example:

      Instructor = (name, ID, dept_name, number_of_instructors)

      - Where is the redundancy in this scheme?
    - Redundant representation of information may lead to data inconsistency among the various copies of information
  - Incompleteness: a bad design may make certain aspects of the enterprise difficult or impossible to model. More later!
- Avoiding bad designs is not enough.
  - There may be a large number of good designs from which we must choose.



Design Approaches

- Entity Relationship Model (covered in this chapter)
  - Models an enterprise as a collection of *entities* and *relationships*
  - Represented diagrammatically by an *entity-relationship diagram:*
- Normalization Theory (Chapter 7)
  - Formalize what designs are bad, and test for them

©Silberschatz, Korth and Sudarshan



Outline of the ER Model



Entity Relationship Model



Entity Relationship (ER) model conceptual level data model proposed by **Peter P Chen in 1970s**

- Design tool
- Graphical representation of the database system
- provides a high-level conceptual data model
- supports the user's perception of the data
- is DBMS and hardware independent
- had many variants
- is composed of entities, attributes, and relationships



The ER model

- The ER data mode was developed to facilitate database design by allowing specification of an **enterprise schema** that represents the overall logical structure of a database.
- The ER data model employs three basic concepts:
  - entity sets,
  - relationship sets,
  - attributes.
- The ER model also has an associated diagrammatic representation, the **ER diagram**, which can express the overall logical structure of a database graphically.

©Silberschatz, Korth and Sudarshan



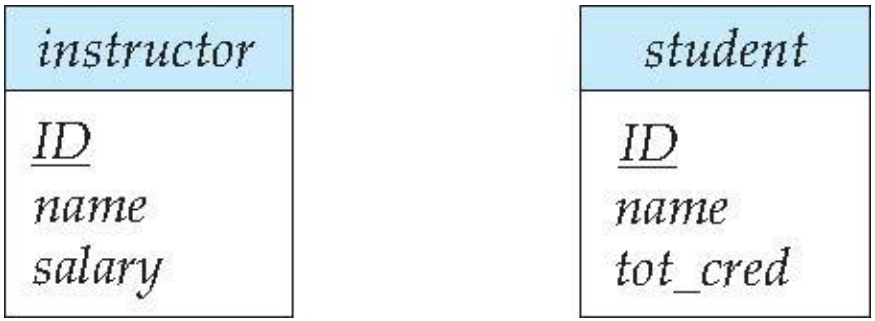
Entity Sets

- An **entity** is an object that exists and is distinguishable from other objects.
  - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
  - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
  - Example:
    - *instructor* = (*ID, name, salary*)
    - *course*= (*course_id, title, credits*)
- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.



Representing Entity sets in ER Diagram

- Rectangles represent entity sets.
- Attributes listed inside entity rectangle
- Underline indicates primary key attributes



 ©Silberschatz, Korth and Sudarshan



Relationship Sets

- A **relationship** is an association among several entities

  Example:

44553 (Peltier)	*advisor*	22222 (Einstein)
student entity	relationship set	*instructor* entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \ldots e_n) \mid e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n\}$$

  where $(e_1, e_2, \ldots, e_n)$ is a relationship

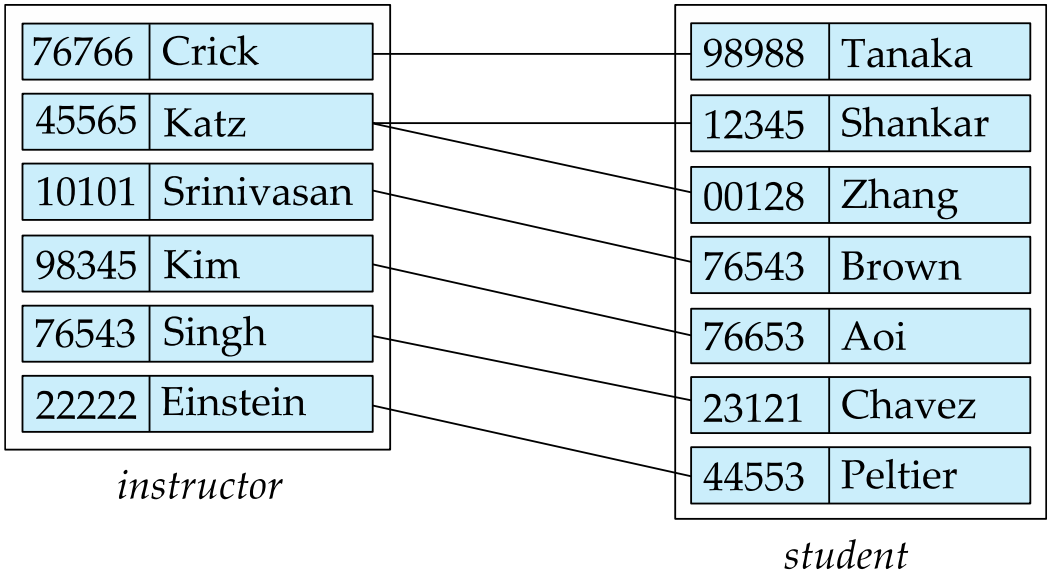
  - Example:

    $(44553,22222) \in advisor$



Relationship Sets (Cont.)

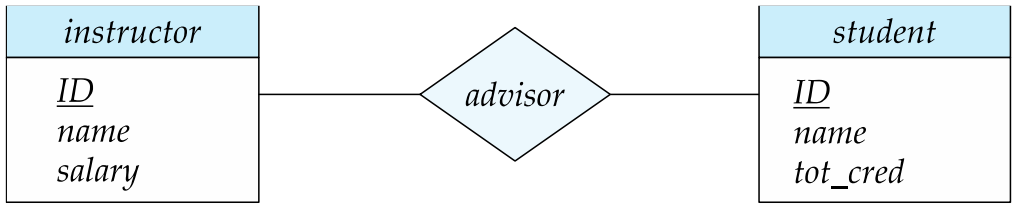
- Example: we define the relationship set *advisor* to denote the associations between students and the instructors who act as their advisors.
- Pictorially, we draw a line between related entities.





Representing Relationship Sets via ER Diagrams

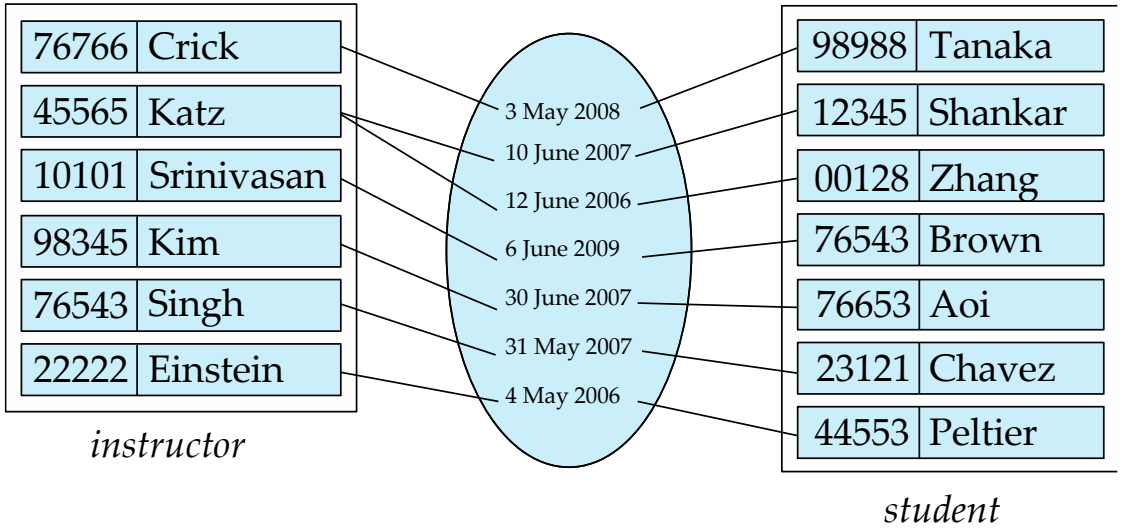
- Diamonds represent relationship sets.





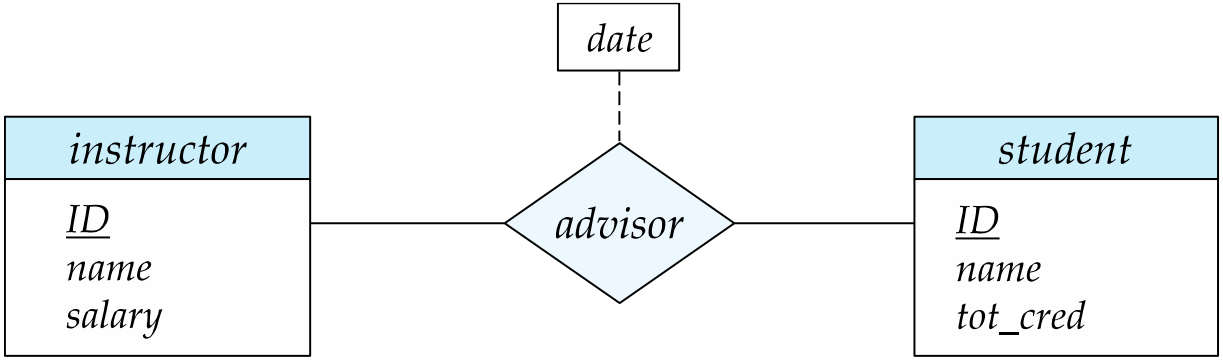
Relationship Sets (Cont.)

- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor





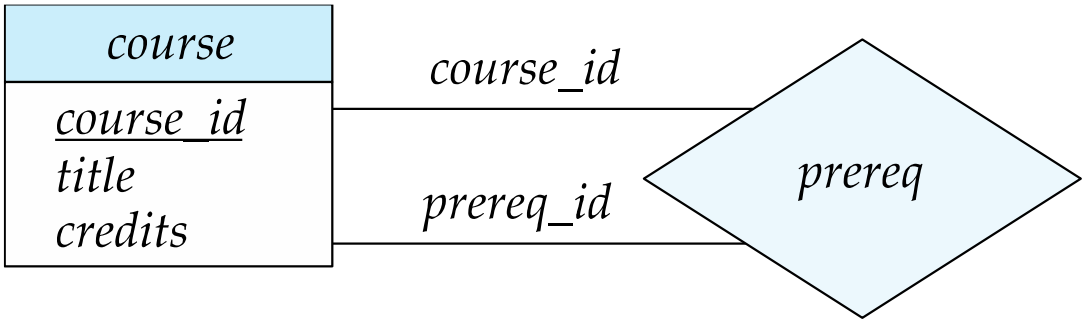
Relationship Sets with Attributes





Roles

- Entity sets of a relationship need not be distinct
  - Each occurrence of an entity set plays a “role” in the relationship
- The labels “*course_id*” and “*prereq_id*” are called **roles**.





Cardinality

- Relationships are rarely one-to-one
- For example, a manager usually manages more than one employee
- This is described by the *cardinality* of the relationship,

- **Cardinality** is the maximum times an entity can relate to an instance with another entity or entity set.
- **Ordinality** is the minimum time an entity can relate to an instance with another entity or entity set. Usually, in ER diagrams, these terms are used under cardinality itself.
- The cardinality constraints are within the maximum and minimum numbers that are relevant to a relationship.
- Cardinality is represented by lines that have different stylings depending on the type of cardinality that is to be shown.



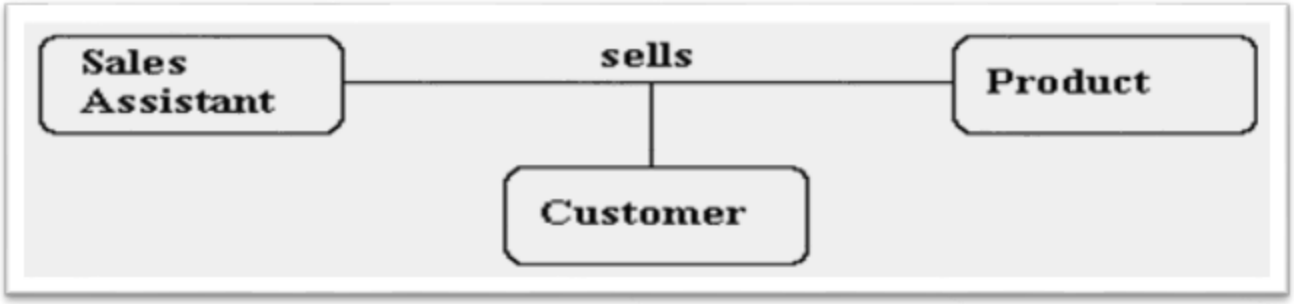
Degree of a Relationship Set

- Binary relationship
  - Involve two entity sets (or degree two).
  - Most relationship sets in a database system are binary.
- Non-binary relationship
  - Example: *students* work on research *projects* under the guidance of an *instructor*.
  - Relationship *proj_guide* is a ternary relationship between *instructor, student,* and *project*



Degree of a Relationship

- The number of participating entities in a relationship is known as the degree of the relationship.
- If there are two entity types involved it is a binary relationship type
- If there are three entity types involved it is a ternary relationship type

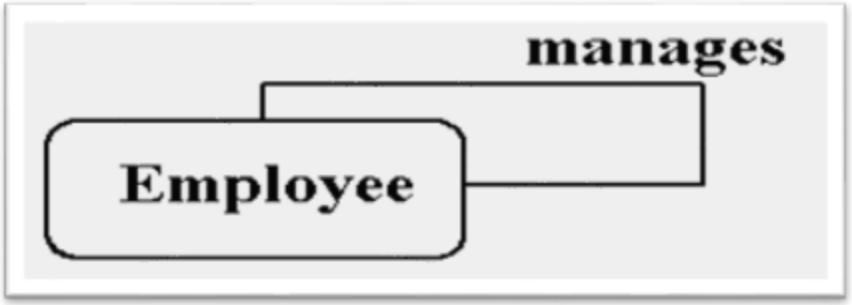


 ©Silberschatz, Korth and Sudarshan



Recursive relationship

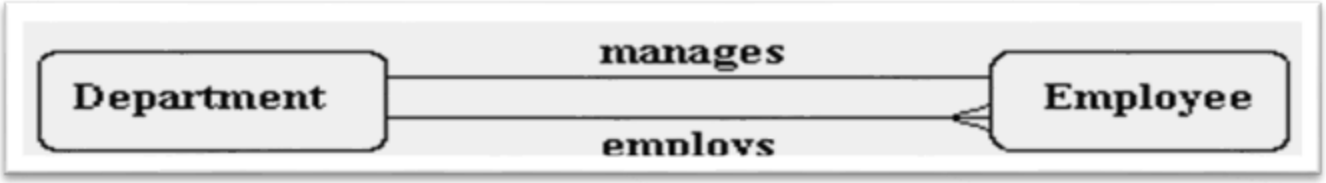
- It is possible to have a n-ary relationship (e.g. quaternary or unary).
- **Unary relationships are also known as a *recursive* relationship**





Degree of a Relationship

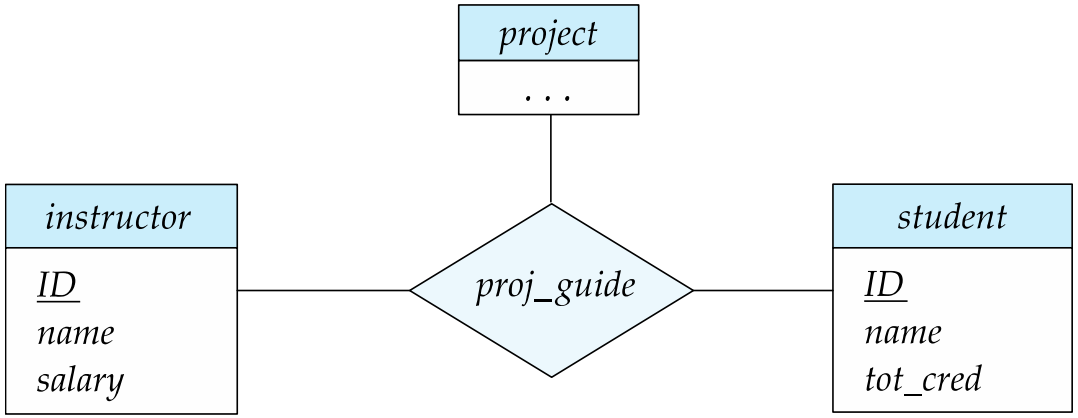
- It is also possible to have entities associated through two or more distinct relationships.





Non-binary Relationship Sets

- E-R Diagram with a Ternary Relationship





Attributes

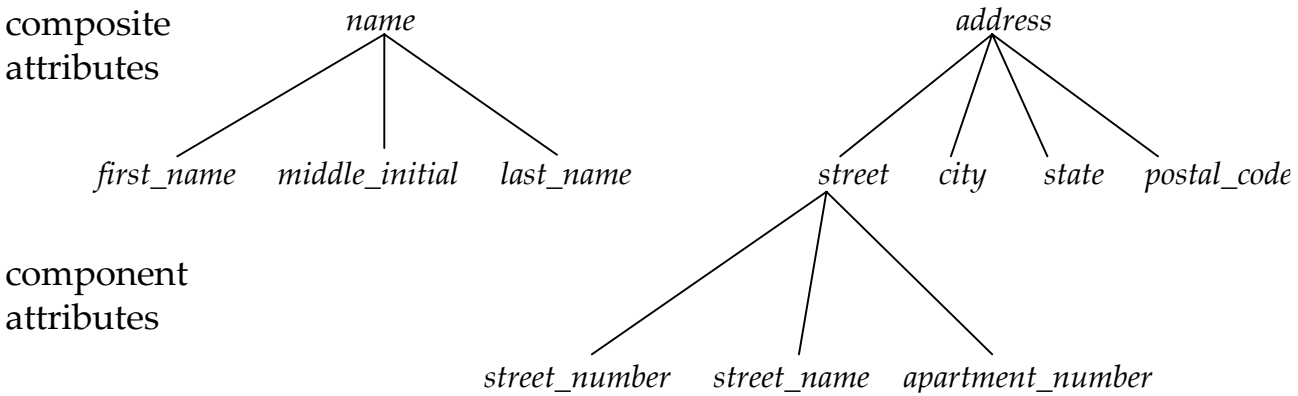
- Attribute types:
  - **Simple** and **composite** attributes.
    - Allow us to divided attributes into subparts (other attributes).
  - **Single-valued** and **multivalued** attributes
    - Example: multivalued attribute: *phone_numbers*
  - **Derived** attributes
    - Can be computed from other attributes
    - Example: age -- can be derived from given date_of_birth
- **Domain** – the set of permitted values for each attribute

©Silberschatz, Korth and Sudarshan



Composite Attributes

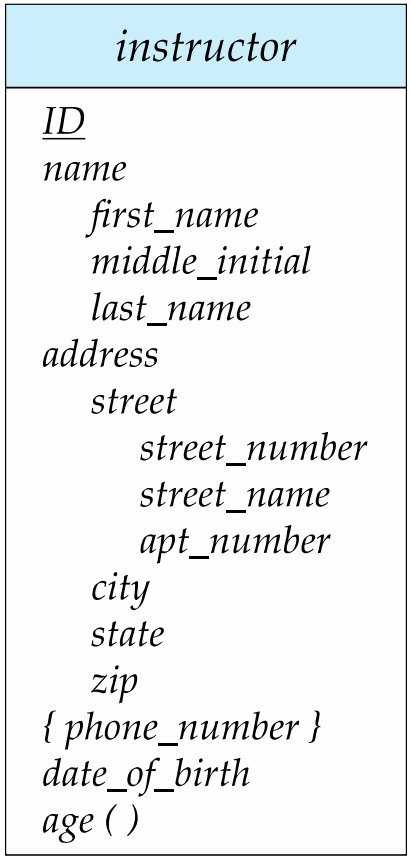
- Composite attributes allow us to divided attributes into subparts (other attributes).



 ©Silberschatz, Korth and Sudarshan



Representing Complex Attributes in ER Diagram



©Silberschatz, Korth and Sudarshan



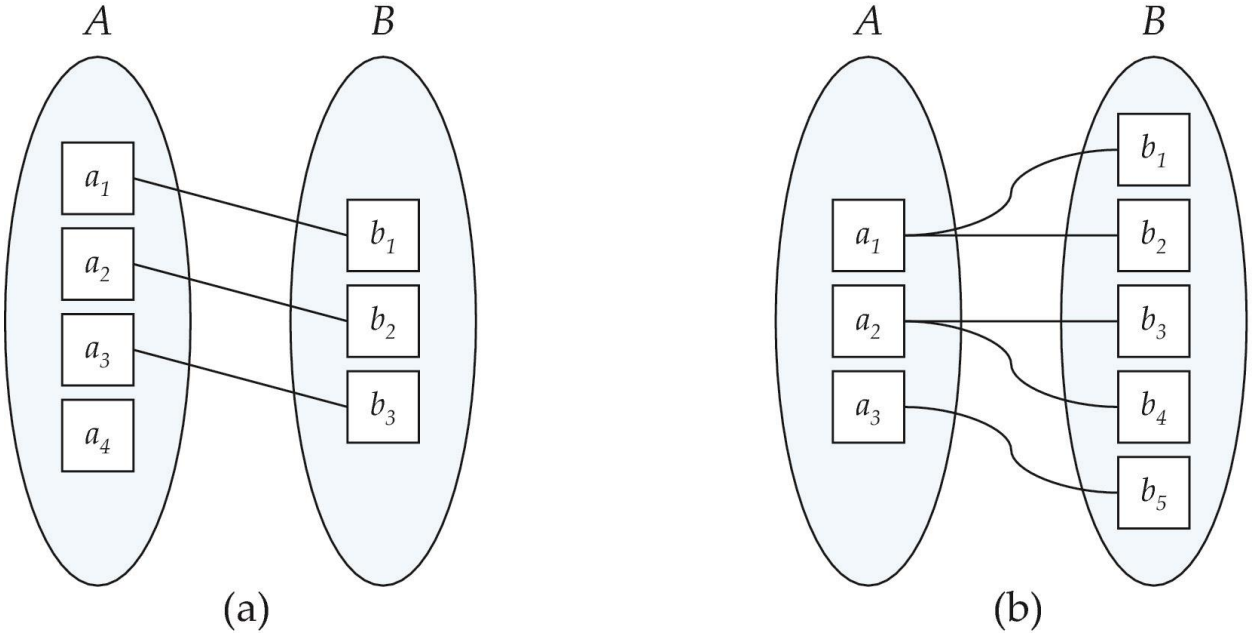
Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
  - One to one
  - One to many
  - Many to one
  - Many to many

©Silberschatz, Korth and Sudarshan



Mapping Cardinalities



One to one

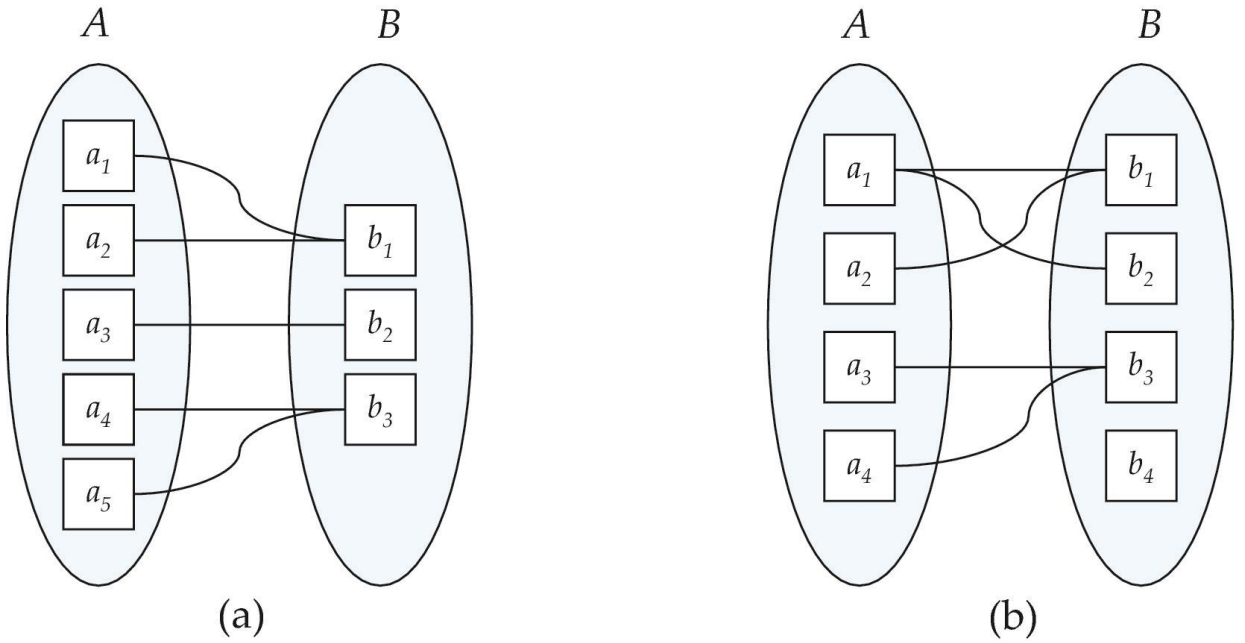
One to many

Note: Some elements in *A* and *B* may not be mapped to any elements in the other set

©Silberschatz, Korth and Sudarshan



Mapping Cardinalities



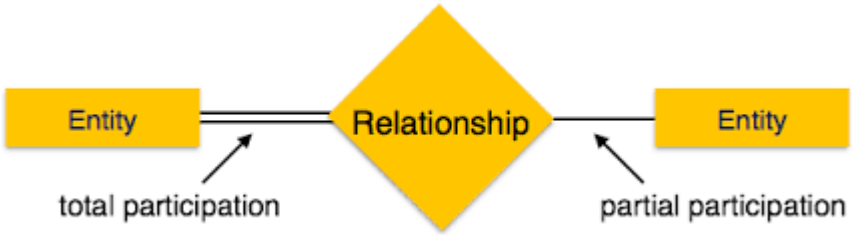
Many to one

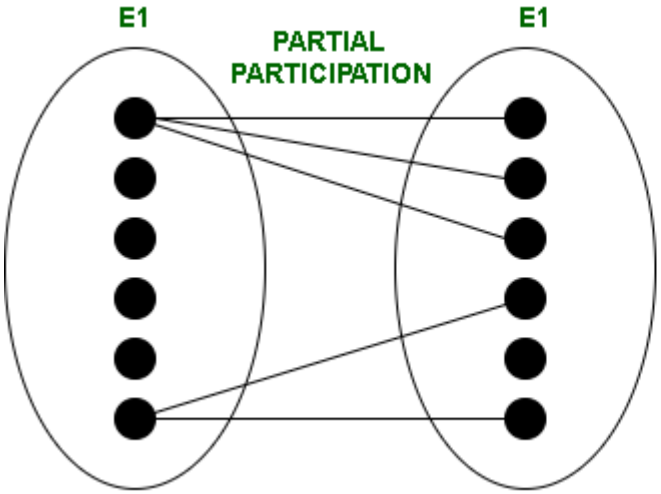
Many to many

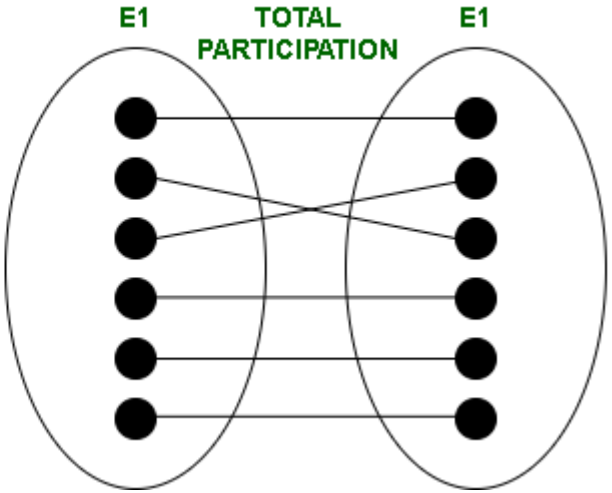
Note: Some elements in A and B may not be mapped to any elements in the other set



Participation



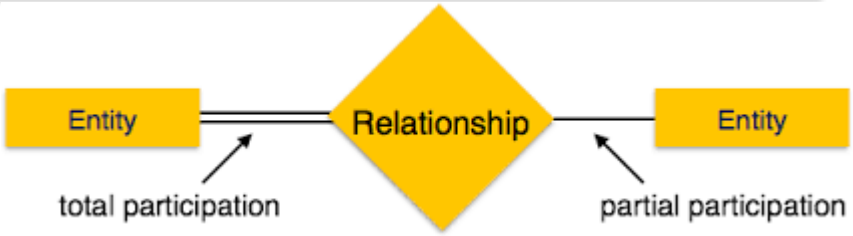




©Silberschatz, Korth and Sudarshan



Participation



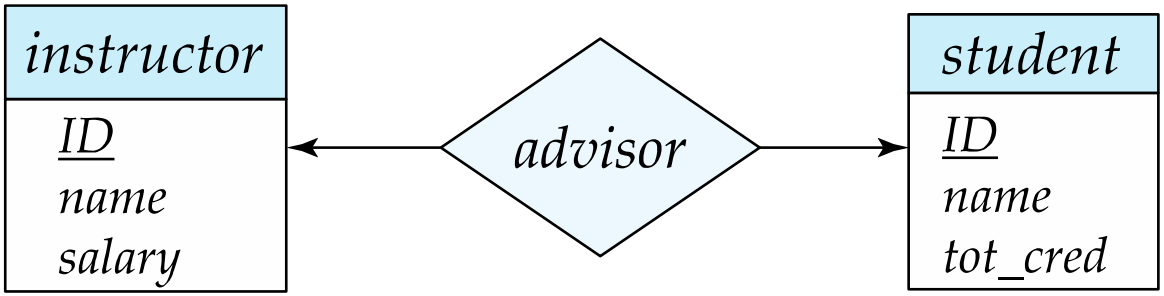
- **Key Differences:**
- **Necessity**:
  - **Total Participation**: The entity must participate in the relationship.
  - **Partial Participation**: The entity may or may not participate in the relationship.
- **Representation**:
  - **Total Participation**: Double line in ER diagrams.
  - **Partial Participation**: Single line in ER diagrams.
- **Summary:**
- **Total Participation** is when every entity in an entity set is involved in a relationship.
- **Partial Participation** is when some entities in an entity set might not be involved in a relationship.

©Silberschatz, Korth and Sudarshan



Cardinality Constraints in ER Diagram

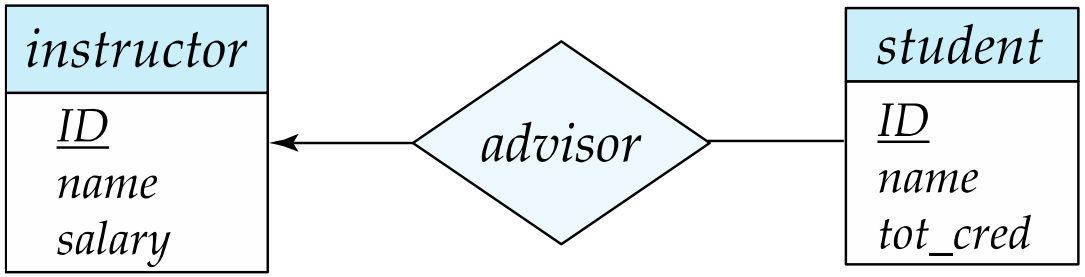
- We express cardinality constraints by drawing either a directed line (→), signifying “one,” or an undirected line (—), signifying “many,” between the relationship set and the entity set.
- One-to-one relationship between an *instructor* and a *student* :
  - A student is associated with at most one *instructor* via the relationship *advisor*
  - A *student* is associated with at most one *department* via *stud_dept*





One-to-Many Relationship

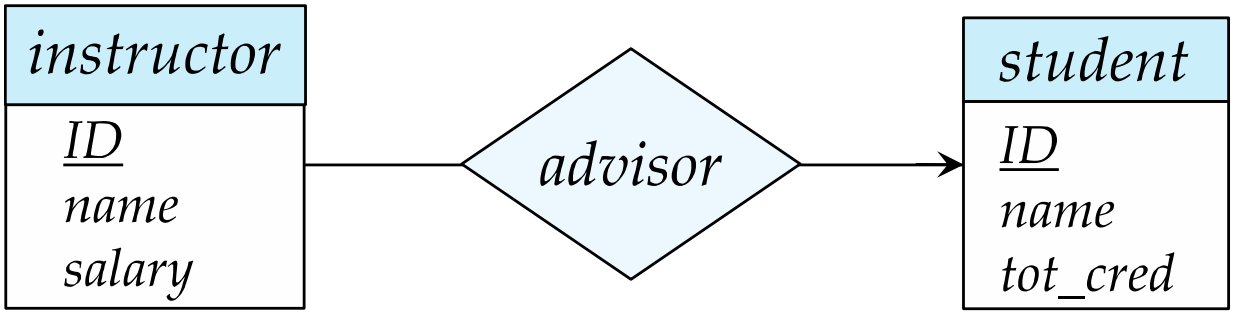
- One-to-many relationship between an *instructor* and a *student*
  - An instructor is associated with several (including 0) students via *advisor*
  - A student is associated with at most one instructor via advisor,





Many-to-One Relationships

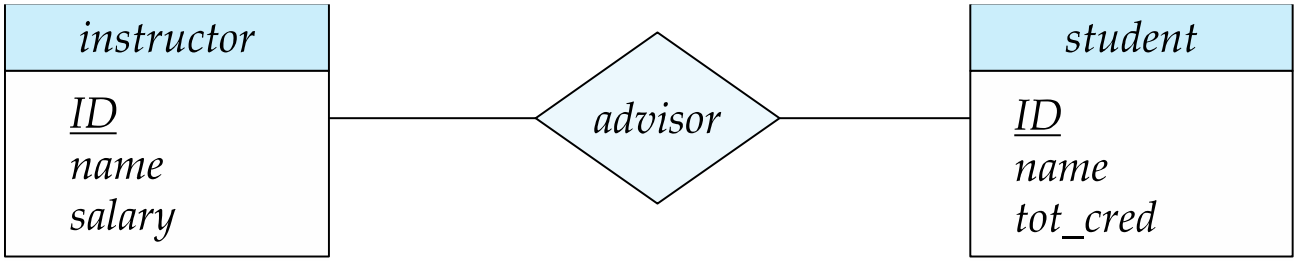
- Many-to-one relationship between an *instructor* and a *student*
  - An instructor is associated with at most one student via *advisor*,
  - A student is associated with several (including 0) instructors via *advisor*





Many-to-Many Relationship

- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*





Total and Partial Participation

- **Total participation** (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set



participation of *student* in *advisor* relation is total

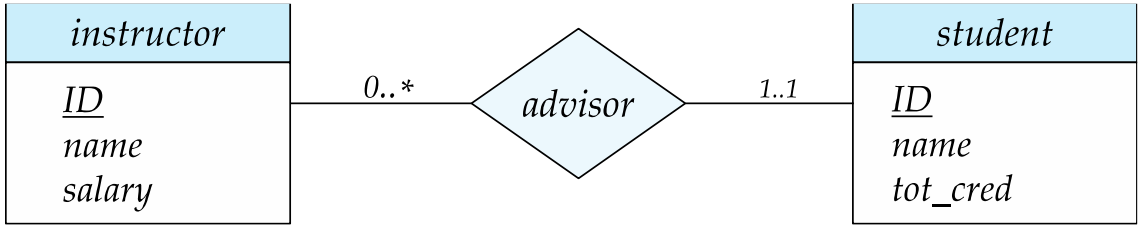
  - Every *student* must have an associated instructor

- **Partial participation**: some entities may not participate in any relationship in the relationship set
  - Example: participation of *instructor* in *advisor* is partial



Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form *l..h*, where *l* is the minimum and *h* the maximum cardinality
  - A minimum value of 1 indicates total participation.
  - A maximum value of 1 indicates that the entity participates in at most one relationship
  - A maximum value of * indicates no limit.
- Example

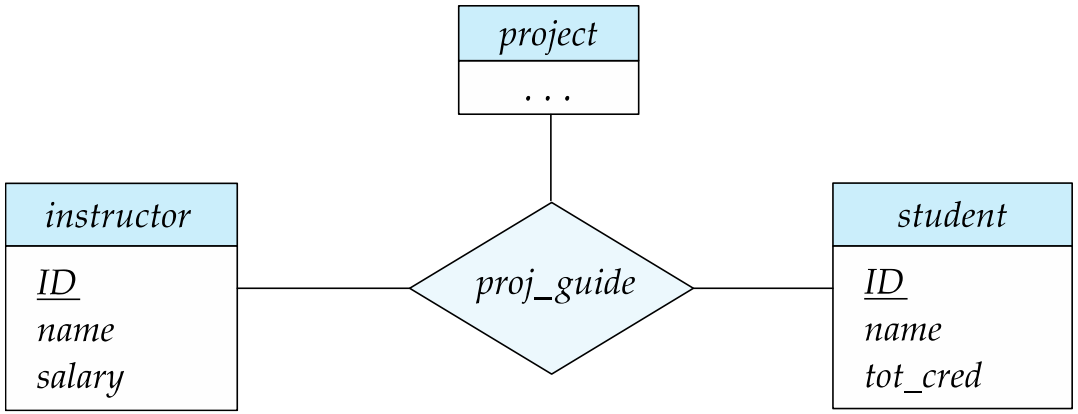


  - Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors



Cardinality Constraints on Ternary Relationship

- We allow at most one arrow out of a ternary (or greater degree) relationship to indicate a cardinality constraint
- For example, an arrow from *proj_guide* to *instructor* indicates each student has at most one guide for a project





Ternary Relationship (Cont.)

- If there is more than one arrow, there are two ways of defining the meaning.
  - For example, a ternary relationship *R* between *A*, *B* and *C* with arrows to *B* and *C* could mean
    1. Each *A* entity is associated with a unique entity from B and *C* or
    2. Each pair of entities from (*A, B*) is associated with a unique *C* entity, and each pair (*A, C*) is associated with a unique *B*
  - Each alternative has been used in different formalisms
  - To avoid confusion, we outlaw more than one arrow



Primary Keys in ER model

- Primary keys provide a way to specify how entities and relations are distinguished. We will consider:
  - Entity sets
  - Relationship sets.
  - Weak entity sets

©Silberschatz, Korth and Sudarshan



Primary key for Entity Sets

- By definition, individual entities are distinct.
- From database perspective, the differences among them must be expressed in terms of their attributes.
- The values of the attribute of an entity must be such that they can uniquely identify the entity.
  - No two entities in an entity set are allowed to have exactly the same value for all attributes.
- A key for an entity is a set of attributes that suffice to distinguish entities from each other

©Silberschatz, Korth and Sudarshan



Primary Key for Relationship Sets

- To distinguish among the various relationships of a relationship set we use the individual primary keys of the entities in the relationship set.
  - Let *R* be a relationship set involving entity sets E_1, E_2, .. E_n
  - The primary key for R consists of the union of the primary keys of entity sets E_1, E_2, ..E_n
  - If the relationship set *R* has attributes a_1, a_2, .., a_m associated with it, then the primary key of *R* also includes the attributes a_1, a_2, .., a_m
- Example: relationship set “advisor”.
  - The primary key consists of *instructor.ID* and *student.ID*
- The choice of the primary key for a relationship set depends on the mapping cardinality of the relationship set.

©Silberschatz, Korth and Sudarshan



Weak Entity Sets

- There are some entities that are totally dependent on other entities.
- For example, consider the entity sets
  - *employee* (ID, name)
  - *children* (name, age)
- The entity set *children* is related to the entity set *employee.* Clearly, without *an employee* entity there will not be a *children* entity.
- The primary key of the *employee* entity set is ID*.*
- *A children* entity set does not have a primary key. The attribute name does not uniquely identify a particular child.
- A **weak entity set** is one whose existence is dependent on another entity set.
- Instead of associating a primary key with a weak entity, we use the identifying entity, along with extra attributes called **discriminator** to uniquely identify a weak entity.



University Example of Weak Entity Sets

- The *section* entities are related to *course* entities and without *course* there will not be *sections.*
- A *section* entity is uniquely identified by a *course_id*, *semester, year*, and *sec_id*.
- The section entities are related to course entities via a relationship set *sec_course*
- The information in *sec_course* is redundant, since *section* already has an attribute *course_id*, which identifies the course with which the section is related.

©Silberschatz, Korth and Sudarshan



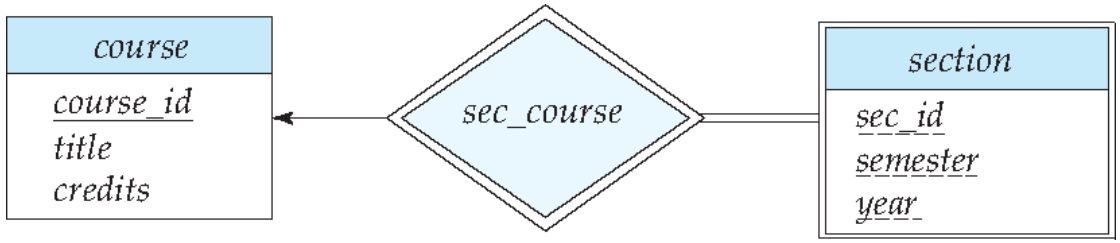
Weak Entity Sets (Cont.)

- An entity set that is not a weak entity set is termed a **strong entity set**.
- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set.
- The identifying entity set is said to **own** the weak entity set that it identifies.
- The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.



Expressing Weak Entity Sets

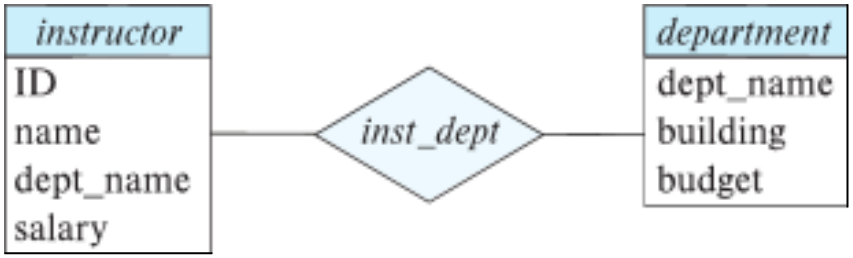
- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* – (*course_id, sec_id, semester, year*)





Redundant Attributes

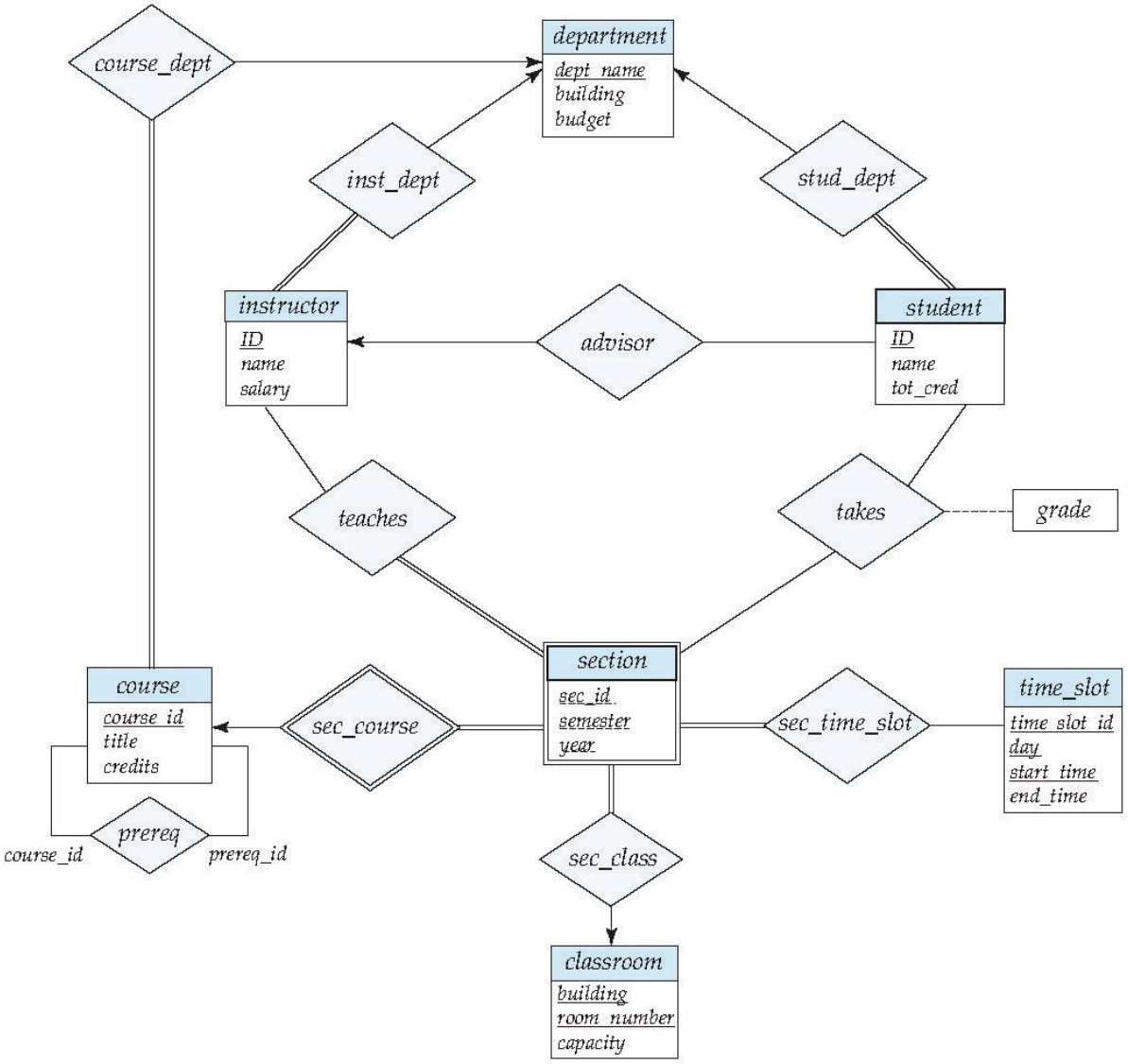
- Suppose we have entity sets:
  - *instructor*, with attributes: *ID*, *name*, *dept_name, salary*
  - *department,* with attributes: *dept_name, building, budget*
- We model the fact that each instructor has an associated department using a relationship set *inst_dept*



- The attribute *dept_name* in *instructor* replicates information present in the relationship and is therefore redundant and needs to be removed.
- BUT: when converting back to tables, in some cases the attribute gets reintroduced, as we will see later.



E-R Diagram for a University Enterprise



 ©Silberschatz, Korth and Sudarshan



Reduction to Relation Schemas



Reduction to Relation Schemas

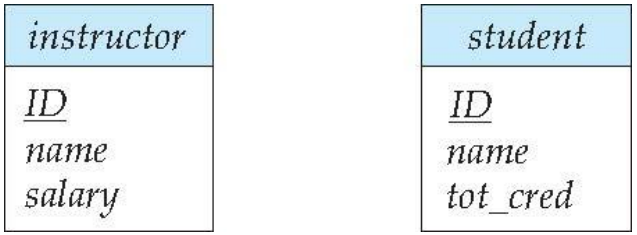
- Entity sets and relationship sets can be expressed uniformly as *relation schemas* that represent the contents of the database.
- A database which conforms to an E-R diagram can be represented by a collection of schemas.
- For each entity set and relationship set there is a unique schema that is assigned the name of the corresponding entity set or relationship set.
- Each schema has a number of columns (generally corresponding to attributes), which have unique names.

©Silberschatz, Korth and Sudarshan



Representing Strong Entity Sets

- A strong entity set reduces to a schema with the same set of attributes



- Example

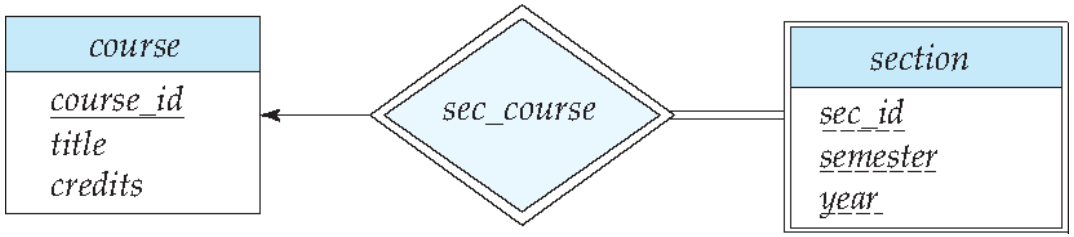
  instructor (*ID*, *name*, *salary*)

  student (*ID*, *name*, *tot_cred*)



Representing Weak Entity Sets

- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set



- Example

section (course_id, sec_id, sem, year)



Example

- A "weak entity type" in a database is an entity that cannot be uniquely identified by its own attributes alone and must rely on a "strong" or "owning" entity to ensure its uniqueness. Weak entities are often used to represent entities that are dependent on another entity.

Example Scenario

Consider a database for a library management system. We can have the following entities:

- **Book** (strong entity)
  - Attributes: `BookID` (Primary Key), `Title`, `Author`, `Publisher`
- **Copy** (weak entity)
  - Attributes: `CopyNumber`, `Condition`
  - Dependent on: `Book`



Weak Entity Type Example

Book Table:

BookID	Title	Author	Publisher
1	"Database Sys"	"Navathe"	"Pearson"
2	"Learn SQL"	"John Smith"	"O'Reilly"

Copy Table (Weak Entity):

BookID	CopyNumber	Condition
1	1	Good
1	2	Fair
2	1	New

Explanation

1.Book is a strong entity with a primary key BookID that uniquely identifies each book in the library.

2.Copy is a weak entity because a CopyNumber alone is not sufficient to identify a specific copy of a book. The uniqueness of a Copy is determined by the combination of BookID and CopyNumber.



ER Diagram

In an ER diagram, this relationship would be represented as follows:

- **Book** (Rectangle)
  - BookID (Primary Key, underlined)
  - Title
  - Author
  - Publisher
- **Copy** (Double Rectangle indicating weak entity)
  - CopyNumber (Partial Key, underlined with a dashed line)
  - Condition
- **Relationship** (Diamond)
  - Between Book and Copy
  - Connected with double lines to Copy (indicating total participation)

©Silberschatz, Korth and Sudarshan



Representation of Composite Attributes

instructor

ID
name
first_name
middle_initial
last_name
address
street
street_number
street_name
apt_number
city
state
zip
{ phone_number }
date_of_birth
age ( )

- Composite attributes are flattened out by creating a separate attribute for each component attribute
- Ignoring multivalued attributes, the extended instructor schema is
  - *instructor(ID, first_name, middle_initial, last_name, street_number, street_name, apt_number, city, state, zip_code, date_of_birth)*



Multivalued Attributes

- A multivalued attribute *M* of an entity set *E* is represented by a separate schema *EM*
- Schema *EM* has attributes corresponding to the primary key of *E* and an attribute corresponding to multivalued attribute *M*
- Example:

  Multivalued attribute *phone_number* of *instructor* is represented by a schema:

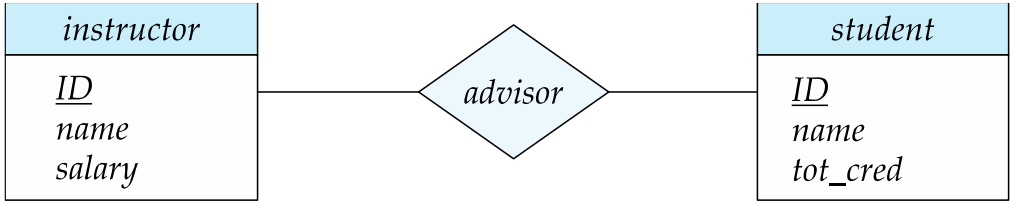
  inst_phone= (*ID*, *phone_number*)
- Each value of the multivalued attribute maps to a separate tuple of the relation on schema *EM*
  - For example, an *instructor* entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples:
    - (22222, 456-7890)
    - (22222, 123-4567)

©Silberschatz, Korth and Sudarshan



Representing Relationship Sets

- A many-to-many relationship set is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- Example: relationship set *advisor*



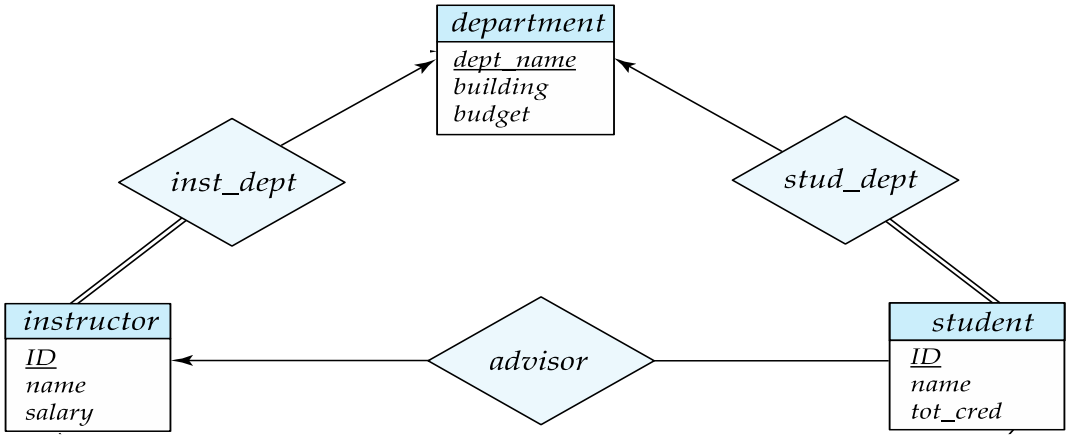
- Corresponding schema:

  advisor = (*s_id, i_id*)



Representing Relationship Sets (Cont.)

- Many-to-one and one-to-many relationship sets that are total on the many-side can be represented by adding an extra attribute to the “many” side, containing the primary key of the “one” side. No schema is created for the relationship set.
- Example:



  - Instead of creating a schema for relationship set *inst_dept*, add an attribute *dept_name* to the schema arising from entity set *instructor*



Representing Relationship Sets (cont.)

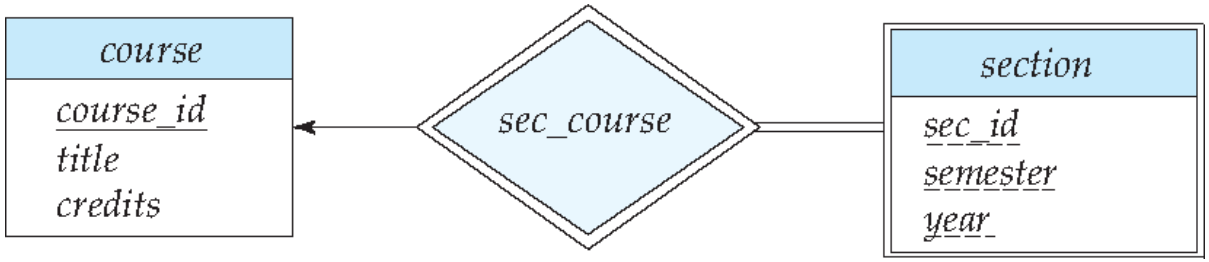
- For one-to-one relationship sets, either side can be chosen to act as the “many” side
  - That is, an extra attribute can be added to either of the tables corresponding to the two entity sets
- If participation is *partial* on the “many” side, replacing a schema by an extra attribute in the schema corresponding to the “many” side could result in null values

©Silberschatz, Korth and Sudarshan



Redundancy of Schemas

- The schema corresponding to a relationship set linking a weak entity set to its identifying strong entity set is redundant and can be eliminated
- Example: The *section* schema already contains the attributes that would appear in the *sec_course* schema





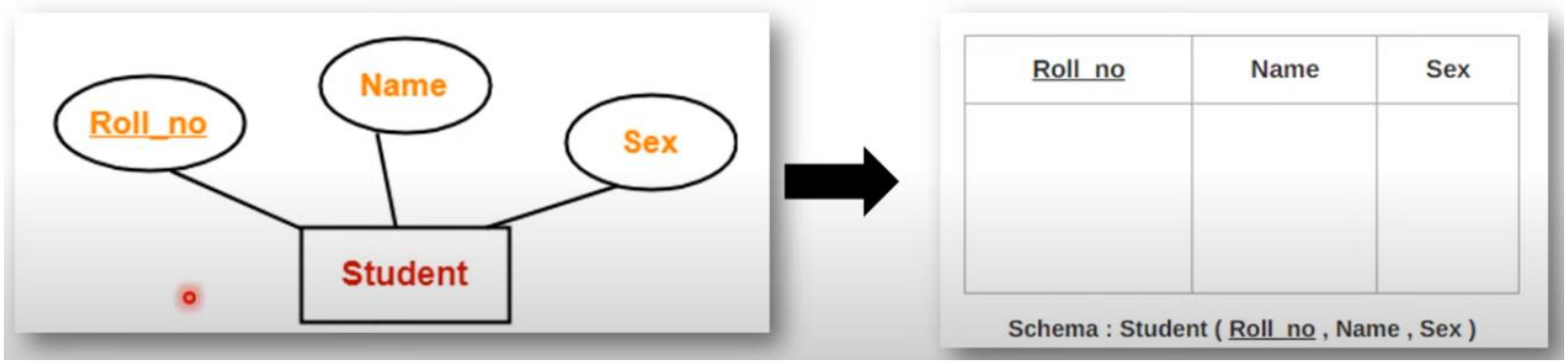
ER to Schema

- Mapping of strong entity types
- Mapping of weak entity types
- Mapping of Binary (1:1) relationship types
- Mapping of Binary (1:N) relationship types
- Mapping of Binary (M:N) relationship types

©Silberschatz, Korth and Sudarshan

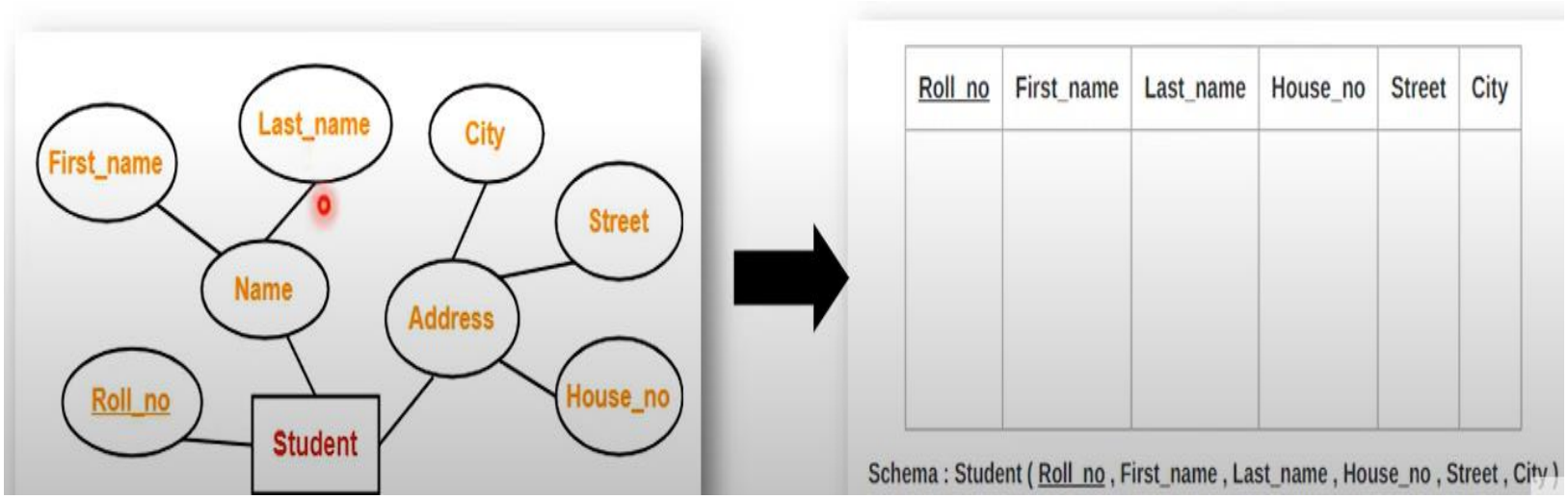


Strong Entity Set with Simple Attributes



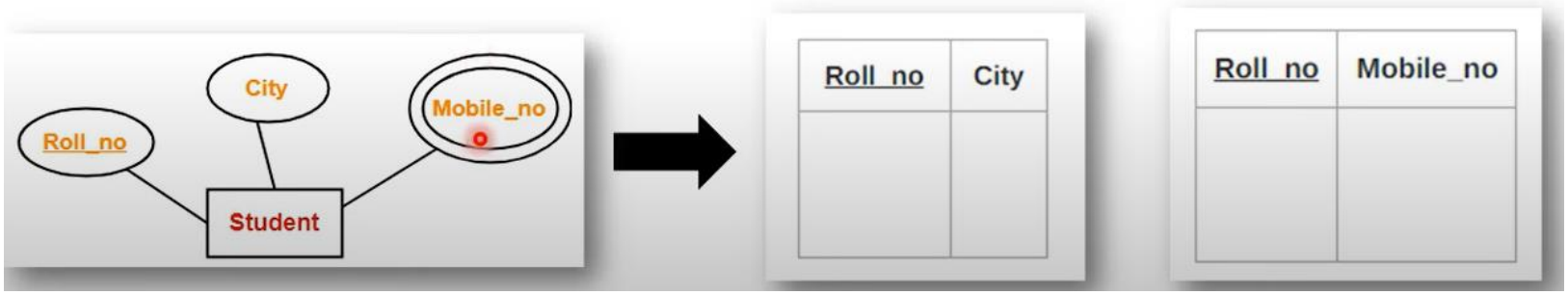


Strong Entity Set with Composite Attributes



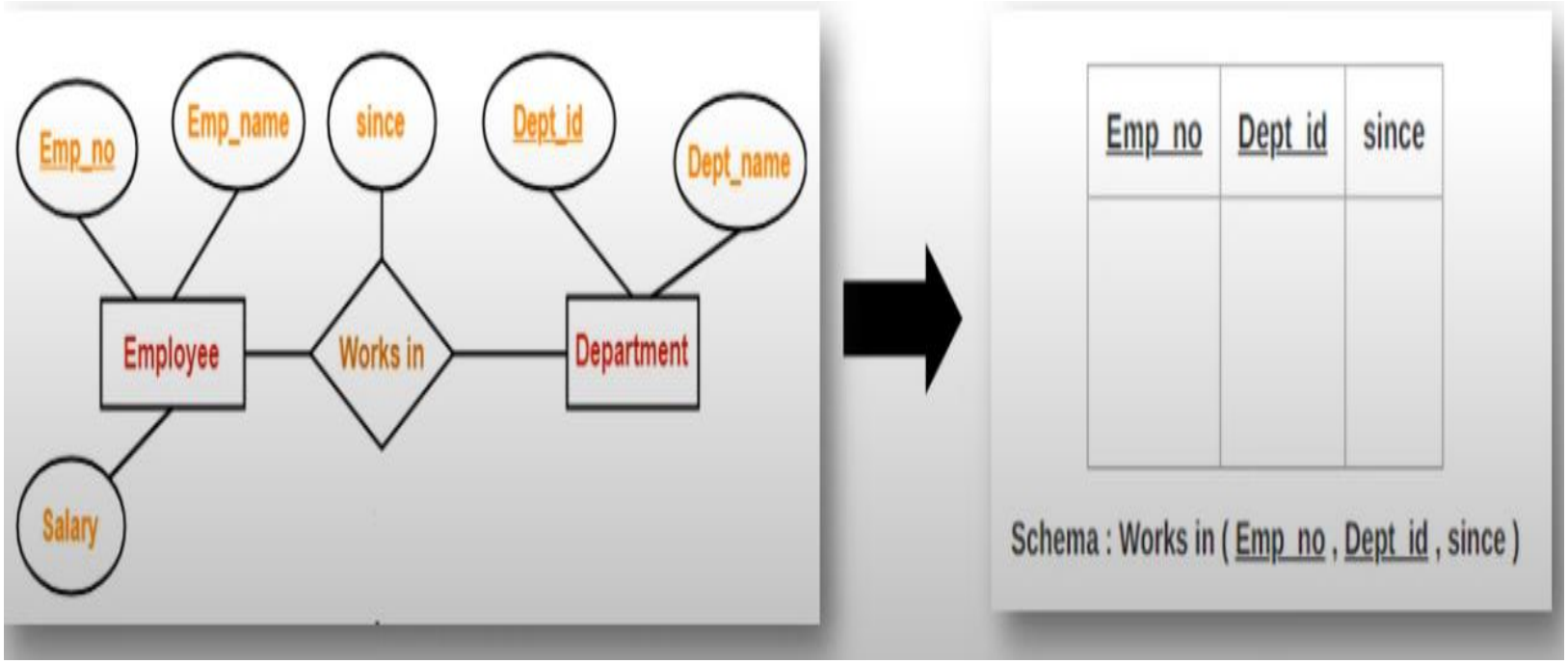


Strong Entity Set with Multivalued Attributes





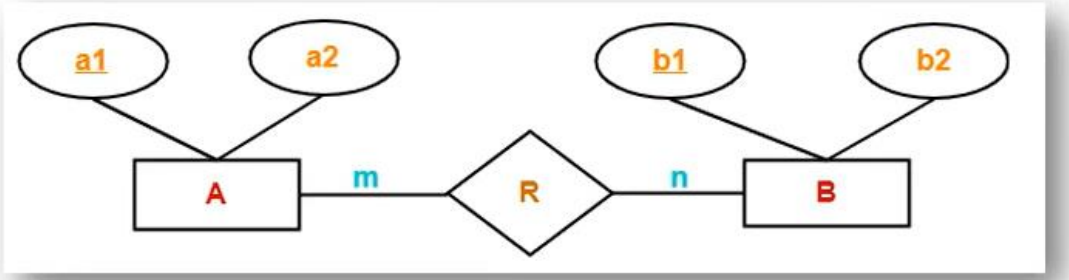
Relationship to Relation



©Silberschatz, Korth and Sudarshan



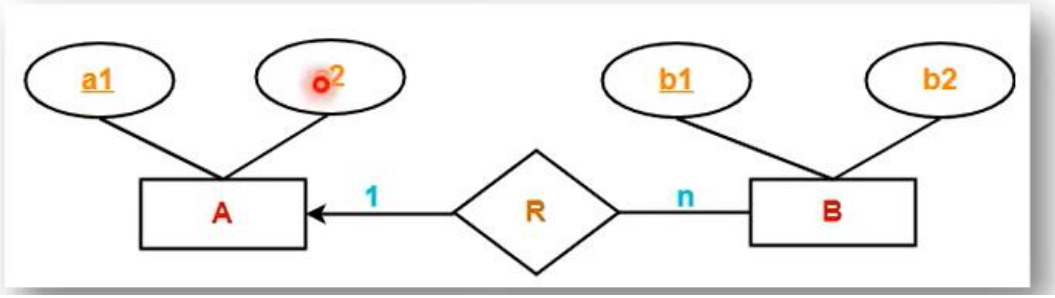
✓Case-1: For Binary Relationship With Cardinality Ratio m:n



Here, three tables will be required-

1. A (a1 , a2)
2. R (a1 , b1)
3. B (b1 , b2)

✓Case-2: For Binary Relationship With Cardinality Ratio 1:n

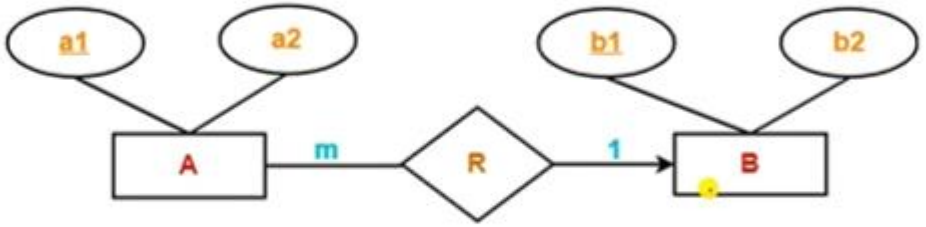


Here, two tables will be required-

1. A (a1 , a2)
2. BR (a1 , b1 , b2)



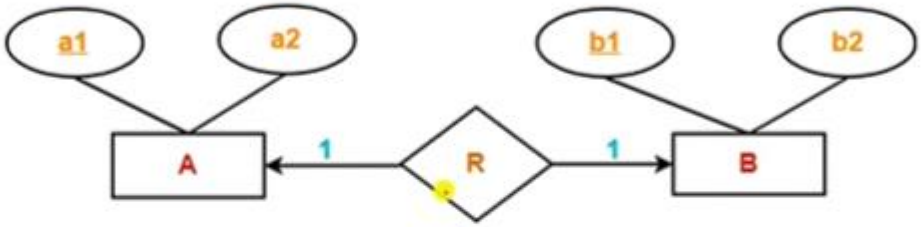
Case-3: For Binary Relationship with Cardinality Ratio m:1



In ***Many-to-One relationship***, **two tables** will be required-

1. **AR (a1 , a2 , b1)**
2. **B (b1 , b2)**

Case-4: For Binary Relationship with Cardinality Ratio 1:1



In ***One-to-One relationship***, **two tables** will be required. Either combine 'R' with 'A' or 'B'

Way-01:

1. **AR (a1 , a2 , b1)**
2. **B (b1 , b2)**

Way-02:

1. **A (a1 , a2)**
2. **BR (b1 , b2, a1)**



Thumb Rules to Remember:

(For determining minimum number of tables)

- **While determining the minimum number of tables required for binary relationships with given cardinality ratios**, following thumb rules must be kept in mind-
  - For binary relationship with cardinality ration **m : n** ,
    - *Separate and individual tables* will be drawn for each entity set and relationship (i.e. **three tables** will be required).
  - For binary relationship with cardinality ratio either **m : 1 or 1 : n** ,
    - Always remember "many side will consume the relationship" i.e. a combined table will be drawn for many side entity set and relationship set (i.e. **Two tables** will be required).
  - For binary relationship with cardinality ratio **1 : 1** ,
    - **Two tables** will be required. You can combine the relationship set with any one of the entity sets.

©Silberschatz, Korth and Sudarshan



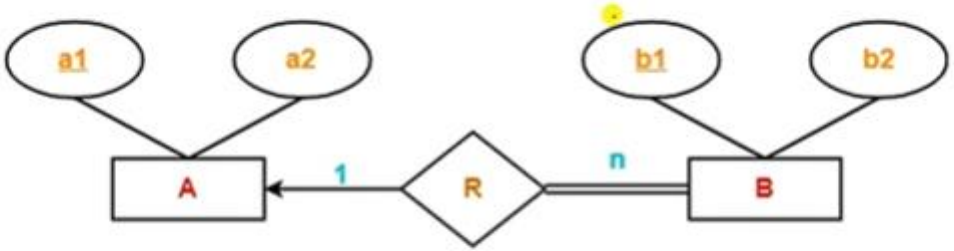
Rule-6: For Binary Relationship with Both Cardinality Constraints and Participation Constraints

- Cardinality constraints will be implemented as discussed in Rule-5.
- Because of the *total participation constraint*, foreign key acquires **NOT NULL** constraint i.e. now foreign key can not be null.
- Two Cases:
  - **Case-1**: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from One Side
  - **Case-2**: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from Both Sides

©Silberschatz, Korth and Sudarshan



Case-1: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from One Side

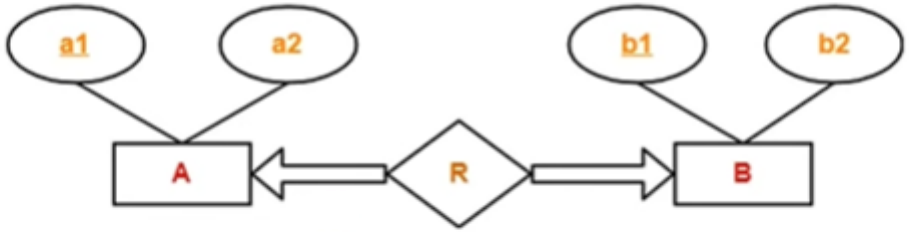


- Because cardinality ratio = 1 : n , we will combine the entity set B and relationship set R.
- Then, **two tables** will be required-
  1. **A (a1 , a2)**
  2. **BR (b1 , b2 , a1)**



Case-2: For Binary Relationship with Cardinality Constraint and Total Participation Constraint from Both Sides

- If there is a key constraint from both the sides of an entity set with total participation, then that binary relationship is represented using **only single table.**



- Here, Only **one table** is required.
  1. **ARB (a1 , a2 , b1 , b2)**



Rule-7: For Binary Relationship with Weak Entity Set

- Weak entity set always appears in association with identifying relationship with **total** participation constraint and there is always **1: n** relationship from identifying entity set to weak entity set.



- Here, **two tables** will be required-
  1. **A (a1 , a2)**
  2. **BR (a1 , b1 , b2)**



Extended E-R Features



Specialization

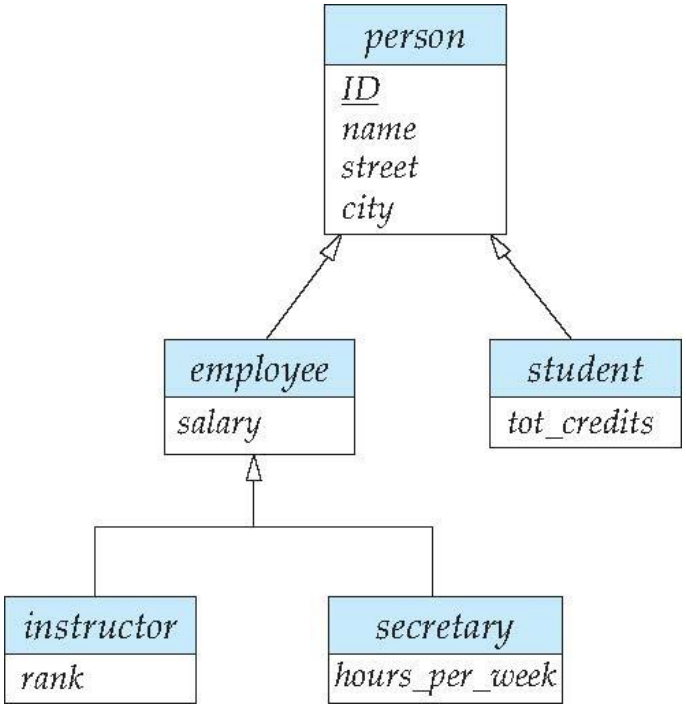
- Top-down design process; we designate sub-groupings within an entity set that are distinctive from other entities in the set.
- These sub-groupings become lower-level entity sets that have attributes or participate in relationships that do not apply to the higher-level entity set.
- Depicted by a *triangle* component labeled ISA (e.g., *instructor* “is a” *person*).
- **Attribute inheritance** – a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.

©Silberschatz, Korth and Sudarshan



Specialization Example

- **Overlapping** – *employee* and *student*
- **Disjoint** – *instructor* and *secretary*
- Total and partial





Representing Specialization via Schemas

- Method 1:
  - Form a schema for the higher-level entity
  - Form a schema for each lower-level entity set, include primary key of higher-level entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

  - Drawback: getting full information about an *employee* requires accessing two relations, the one corresponding to the low-level schema and the one corresponding to the high-level schema

©Silberschatz, Korth and Sudarshan



Representing Specialization as Schemas (Cont.)

- Method 2:
  - Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

  - Drawback: *name, street* and *city* may be stored redundantly for people who are both students and employees

©Silberschatz, Korth and Sudarshan



Generalization

- **A bottom-up design process** – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.



Completeness constraint

- Specifies whether an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization.
  - **total**: an entity must belong to one of the lower-level entity sets
  - **partial**: an entity need not belong to one of the lower-level entity sets



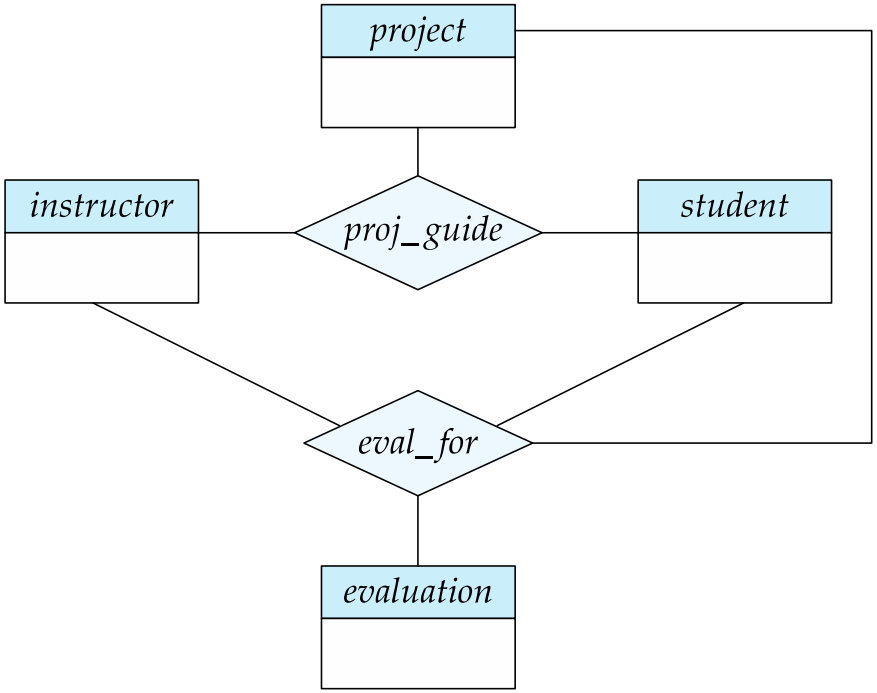
Completeness constraint (Cont.)

- Partial generalization is the default.
- We can specify total generalization in an ER diagram by adding the keyword **total** in the diagram and drawing a dashed line from the keyword to the corresponding hollow arrow-head to which it applies (for a total generalization), or to the set of hollow arrow-heads to which it applies (for an overlapping generalization).
- The *student* generalization is total: All student entities must be either graduate or undergraduate. Because the higher-level entity set arrived at through generalization is generally composed of only those entities in the lower-level entity sets, the completeness constraint for a generalized higher-level entity set is usually total



Aggregation

- Consider the ternary relationship *proj_guide*, which we saw earlier
- Suppose we want to record evaluations of a student by a guide on a project





Aggregation (Cont.)

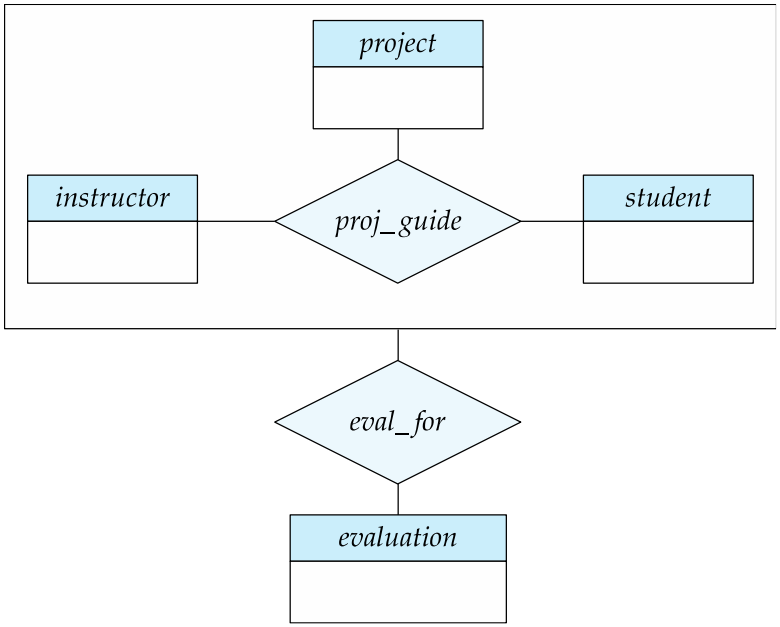
- Relationship sets *eval_for* and *proj_guide* represent overlapping information
  - Every *eval_for* relationship corresponds to a *proj_guide* relationship
  - However, some *proj_guide* relationships may not correspond to any *eval_for* relationships
    - So we can't discard the *proj_guide* relationship
- Eliminate this redundancy via *aggregation*
  - Treat relationship as an abstract entity
  - Allows relationships between relationships
  - Abstraction of relationship into new entity

©Silberschatz, Korth and Sudarshan



Aggregation (Cont.)

- Eliminate this redundancy via *aggregation* without introducing redundancy, the following diagram represents:
  - A student is guided by a particular instructor on a particular project
  - A student, instructor, project combination may have an associated evaluation



 ©Silberschatz, Korth and Sudarshan



Aggregation -- Reduction to Relational Schemas

- To represent aggregation, create a schema containing
  - Primary key of the aggregated relationship,
  - The primary key of the associated entity set
  - Any descriptive attributes
- In our example:
  - The schema *eval_for* is:

    eval_for (*s_ID, project_id, i_ID, evaluation_id*)
  - The schema *proj_guide* is redundant.

©Silberschatz, Korth and Sudarshan

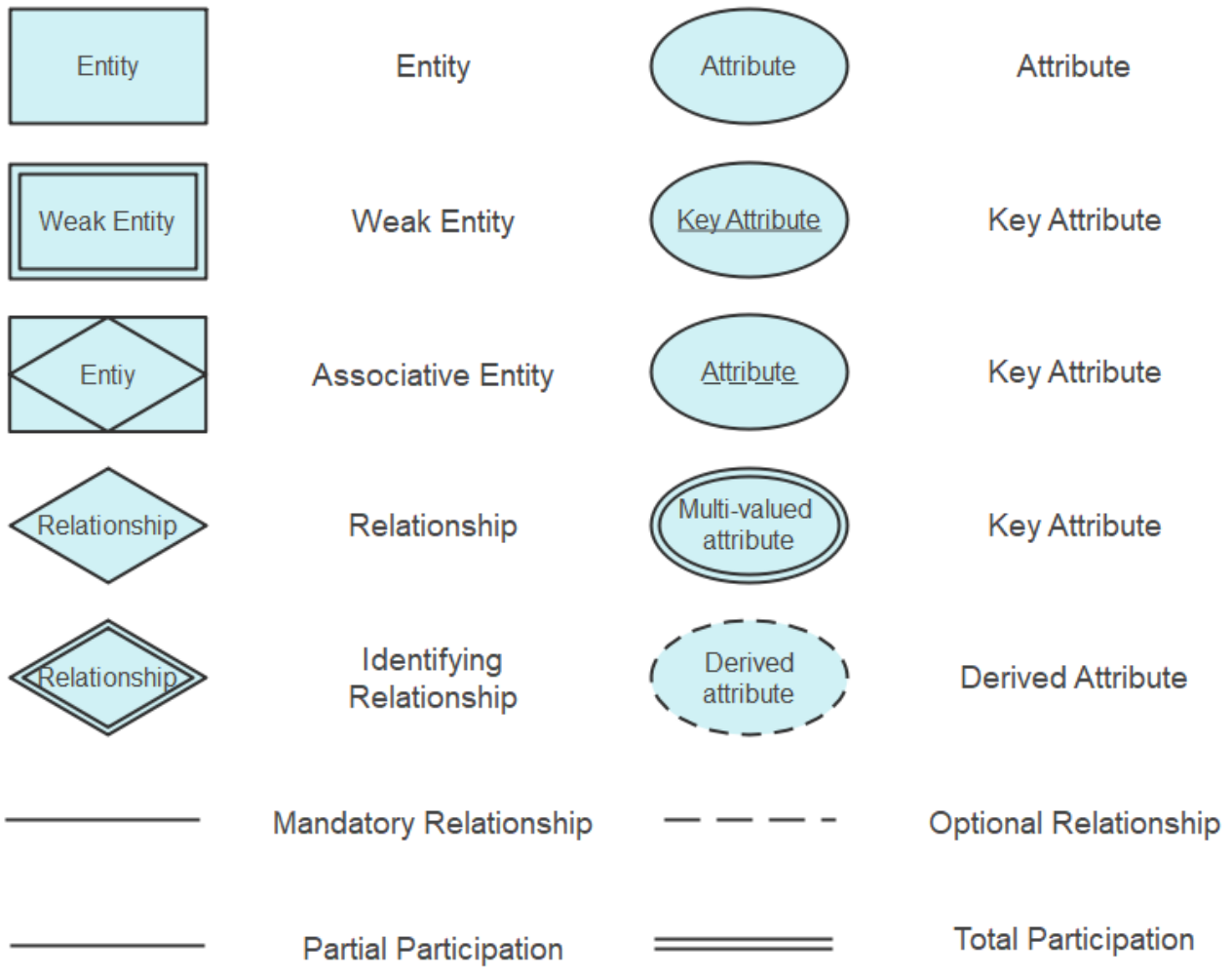


E-R Design Decisions

- The use of an attribute or entity set to represent an object.
- Whether a real-world concept is best expressed by an entity set or a relationship set.
- The use of a ternary relationship versus a pair of binary relationships.
- The use of a strong or weak entity set.
- The use of specialization/generalization – contributes to modularity in the design.
- The use of aggregation – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.



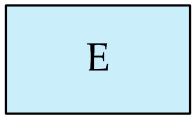
ERD Symbols



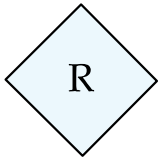
 ©Silberschatz, Korth and Sudarshan



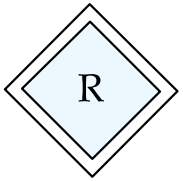
Summary of Symbols Used in E-R Notation



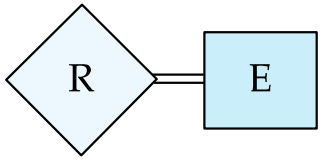
entity set



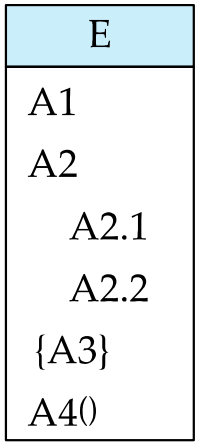
relationship set



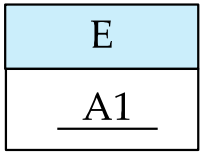
identifying relationship set for weak entity set



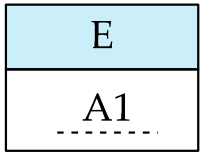
total participation of entity set in relationship



attributes:
simple (A1),
composite (A2) and
multivalued (A3)
derived (A4)



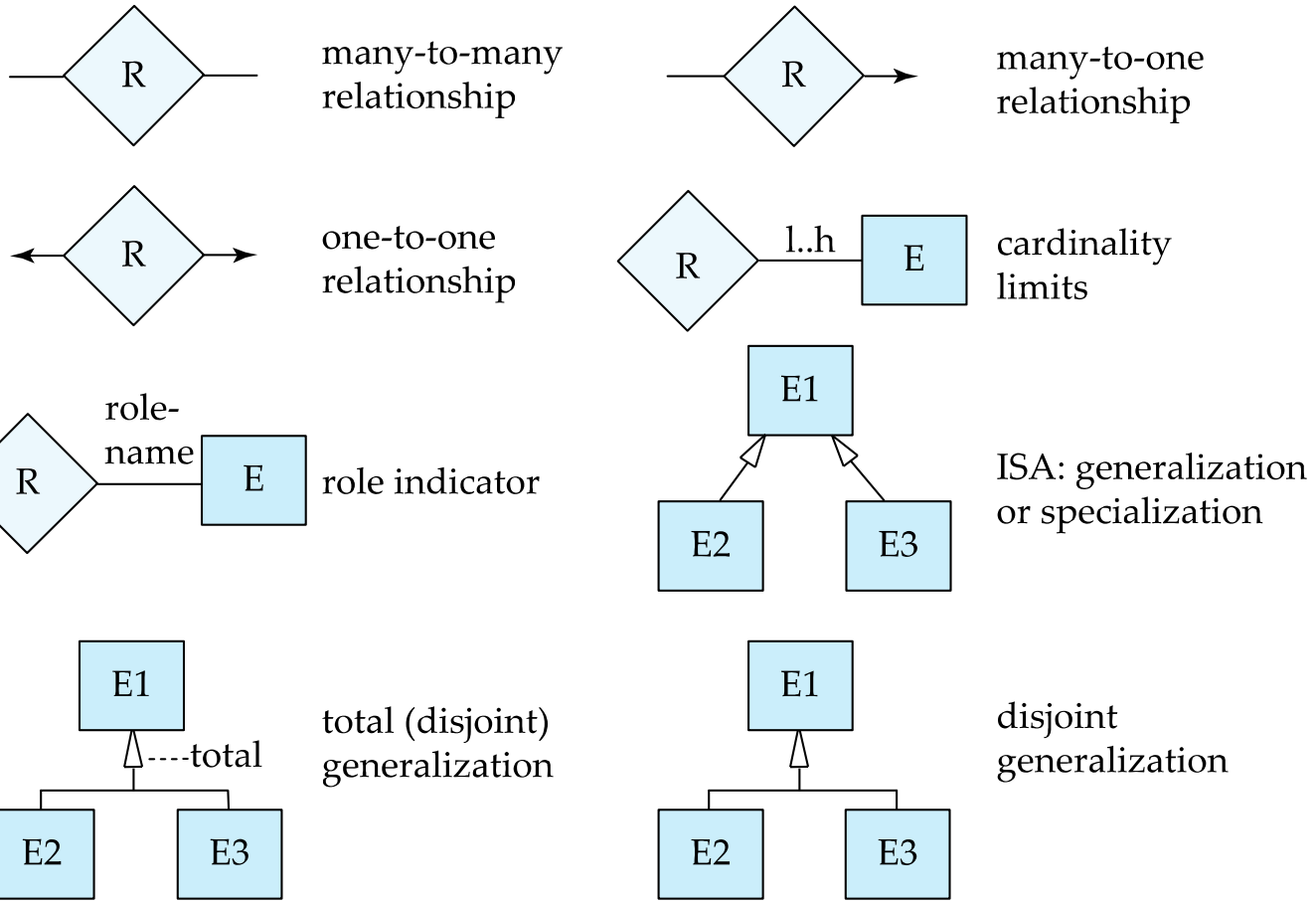
primary key



discriminating attribute of weak entity set



Symbols Used in E-R Notation (Cont.)



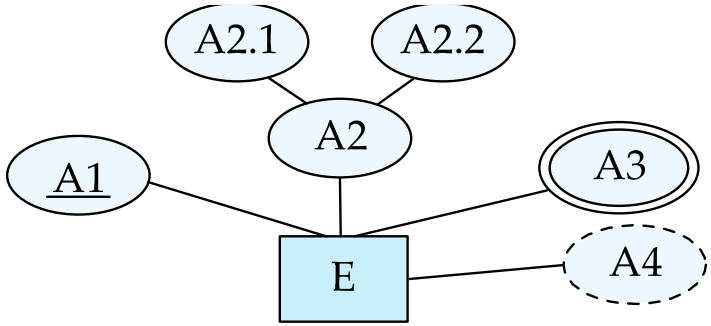
 ©Silberschatz, Korth and Sudarshan

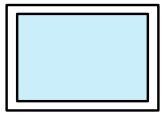


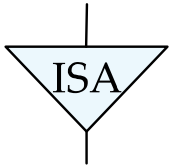
Alternative ER Notations

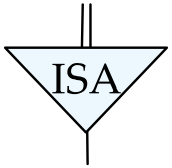
- Chen, IDE1FX, …

entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



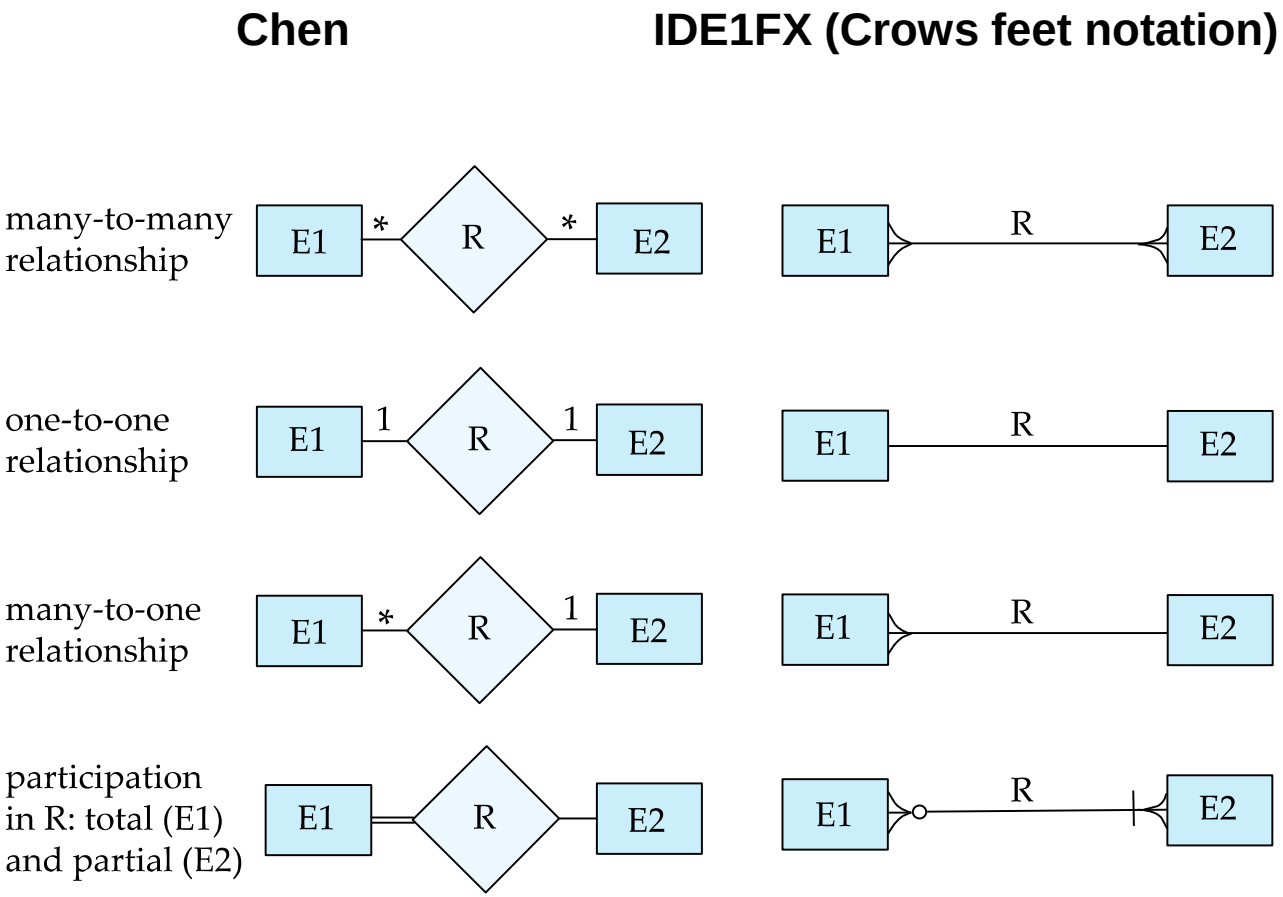
weak entity set 

generalization 

total generalization 



Alternative ER Notations (Cont.)



 ©Silberschatz, Korth and Sudarshan



Relationship

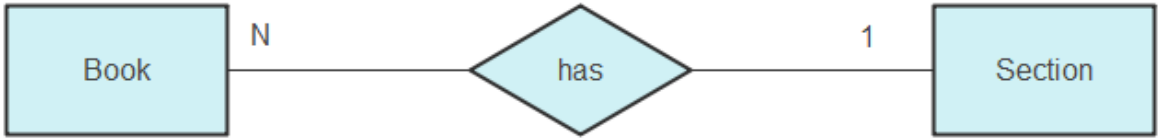
one-to-one (1:1)



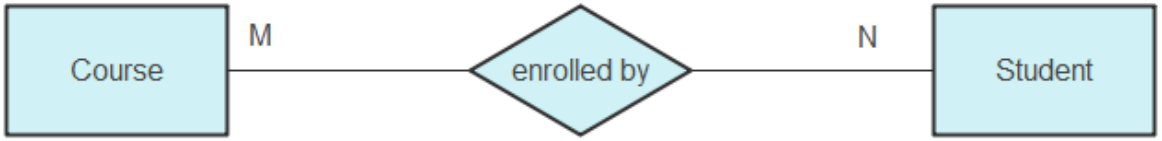
one-to-many (1:N)

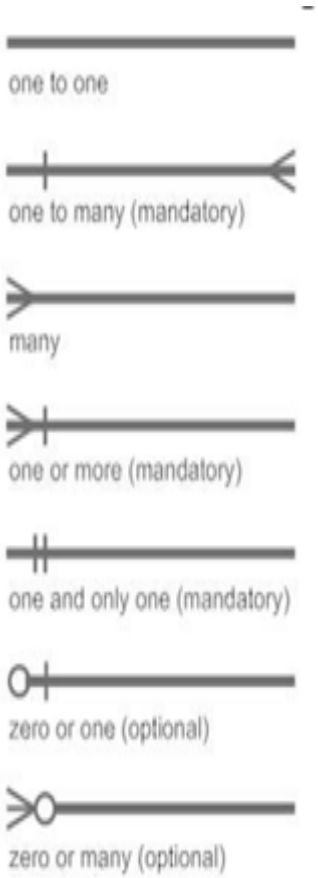


many-to-one (N:1)



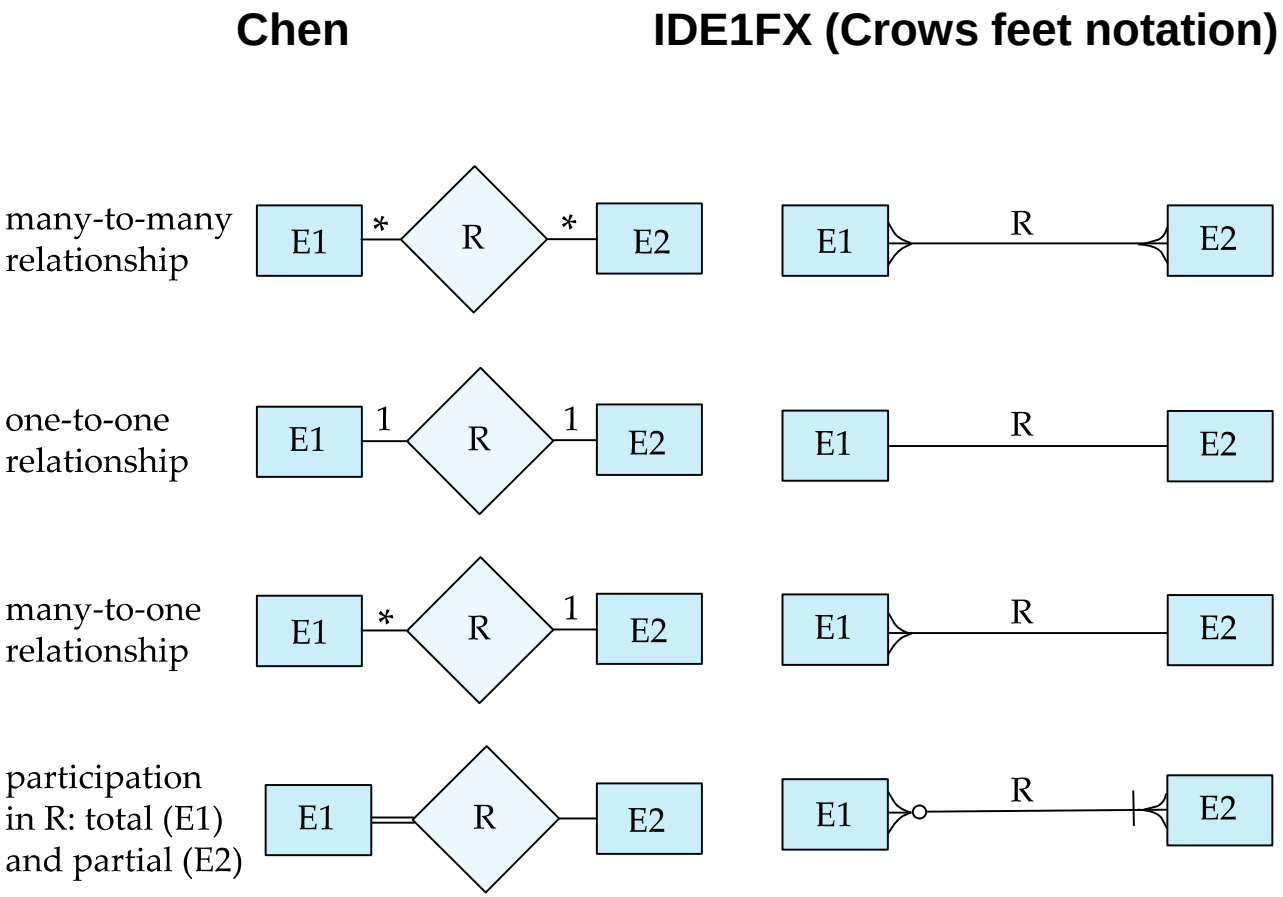
many-to-many (M:N)







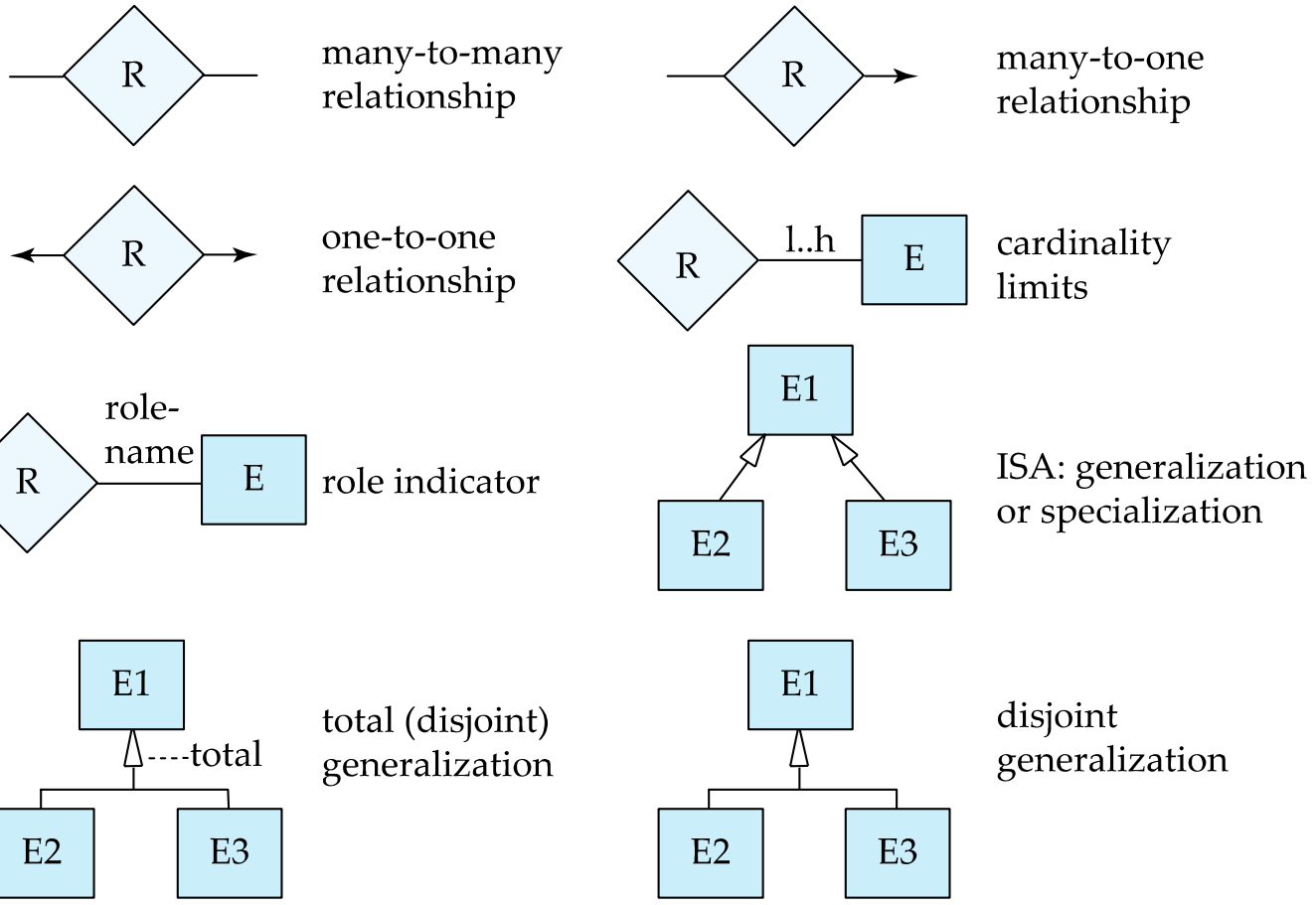
Alternative ER Notations (Cont.)



 ©Silberschatz, Korth and Sudarshan



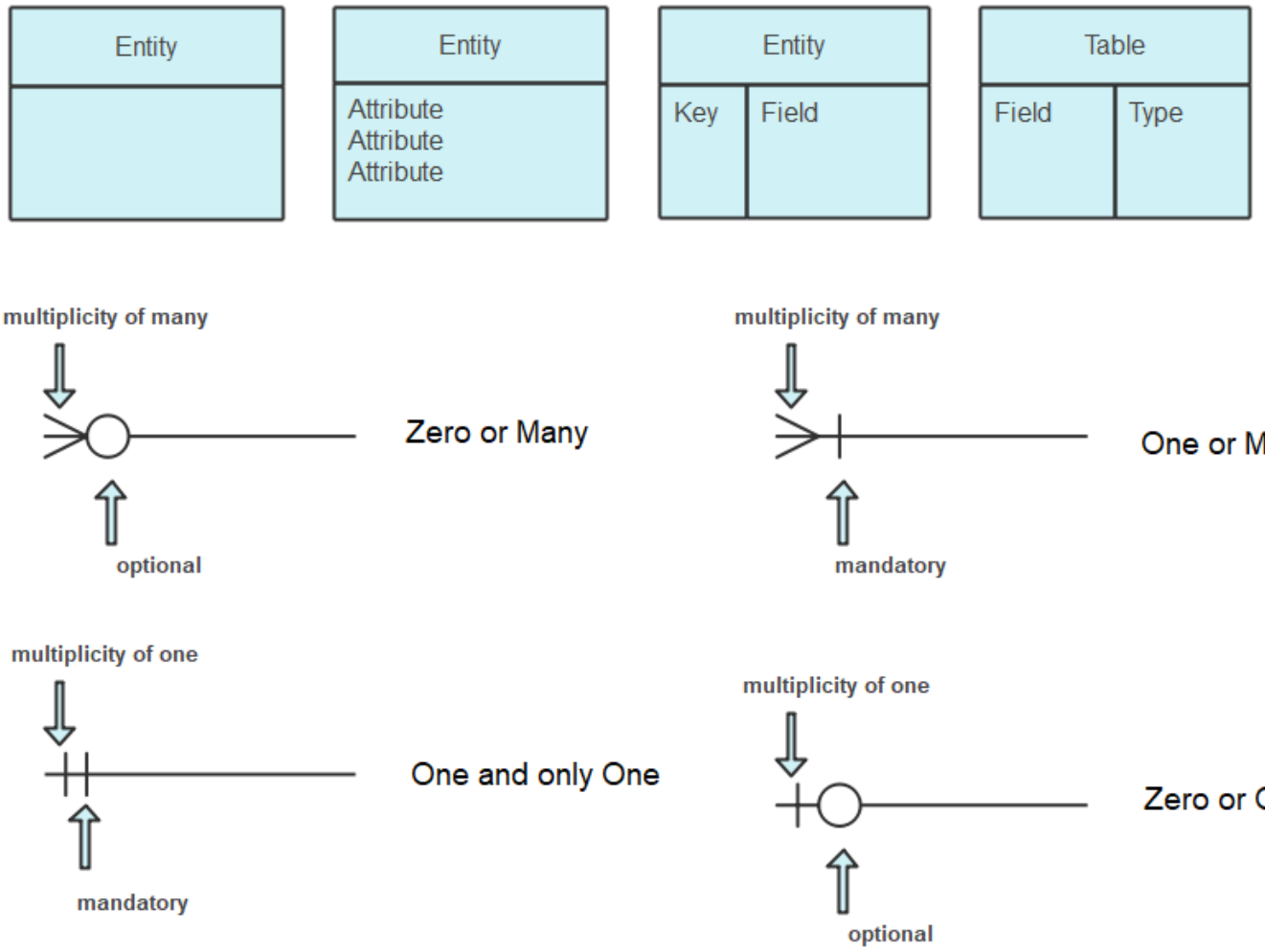
Symbols Used in E-R Notation (Cont.)



 ©Silberschatz, Korth and Sudarshan



Crow's Foot Notations

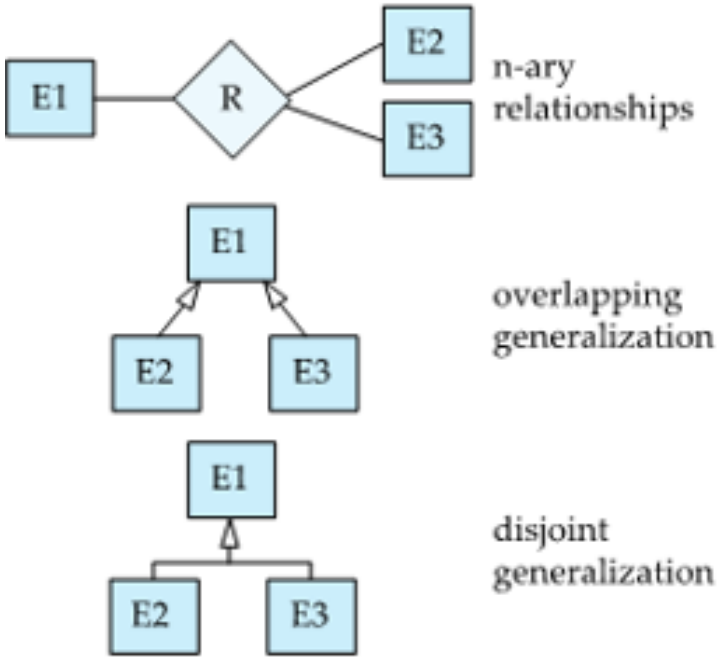


©Silberschatz, Korth and Sudarshan



ER

ER Diagram Notation



* Generalization can use merged or separate arrows independent of disjoint/overlapping



Country Bus Company

A Country Bus Company owns a number of busses.

- Each bus is allocated to a particular route, although some routes may
- have several busses. Each route passes through a number of towns.
- One or more drivers are allocated to each stage of a route, which corresponds to a journey through some or all of the towns on a route.
- Some of the towns have a garage where busses are kept and each of the busses are identified by the registration number and can carry different numbers of passengers, since the vehicles vary in size and can be single or double-decked.
- Each route is identified by a route number and information is available on the average number of passengers carried per day for each route.
- Drivers have an employee number, name, address, and sometimes a telephone number.

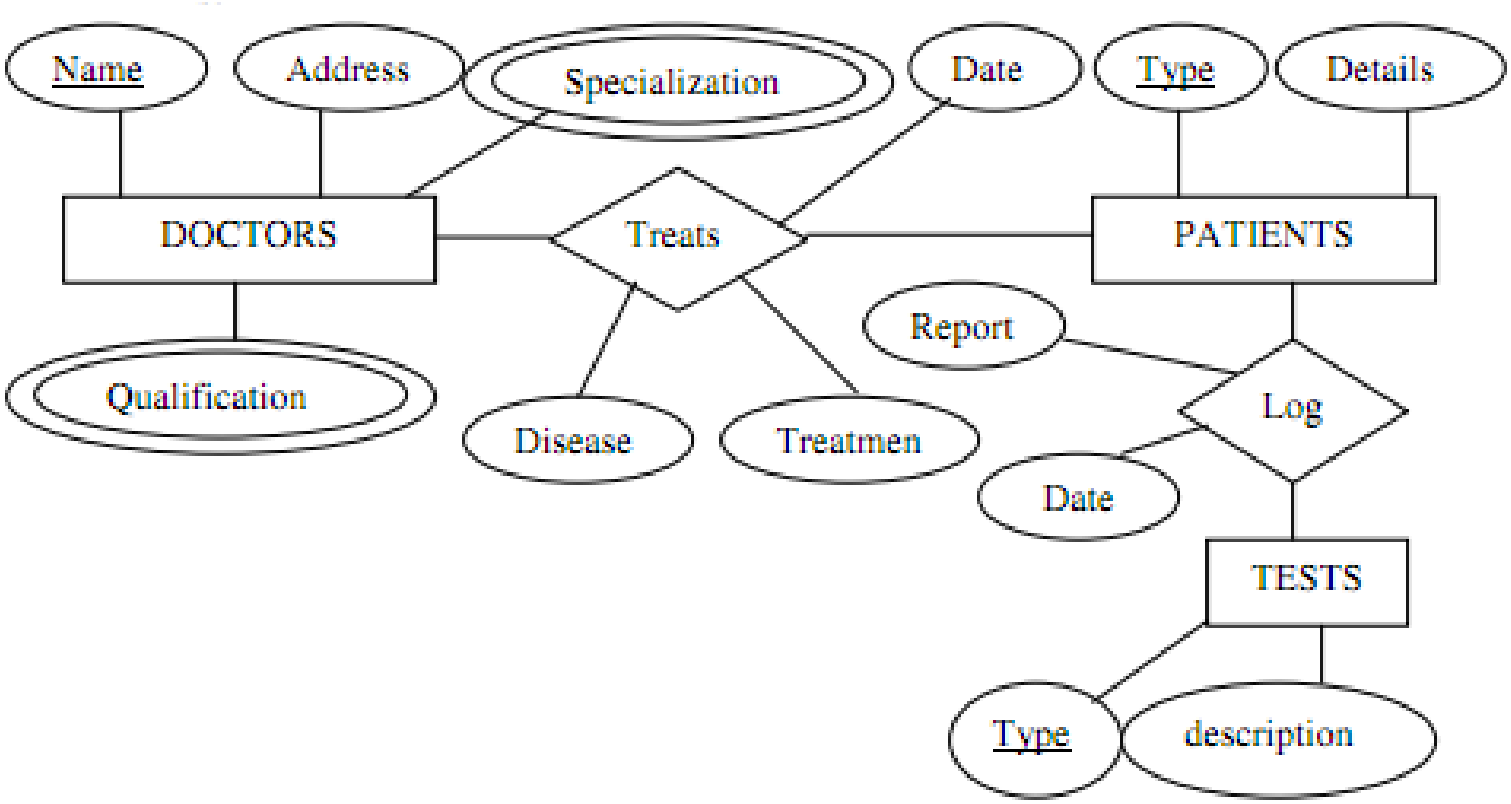
©Silberschatz, Korth and Sudarshan



Entities

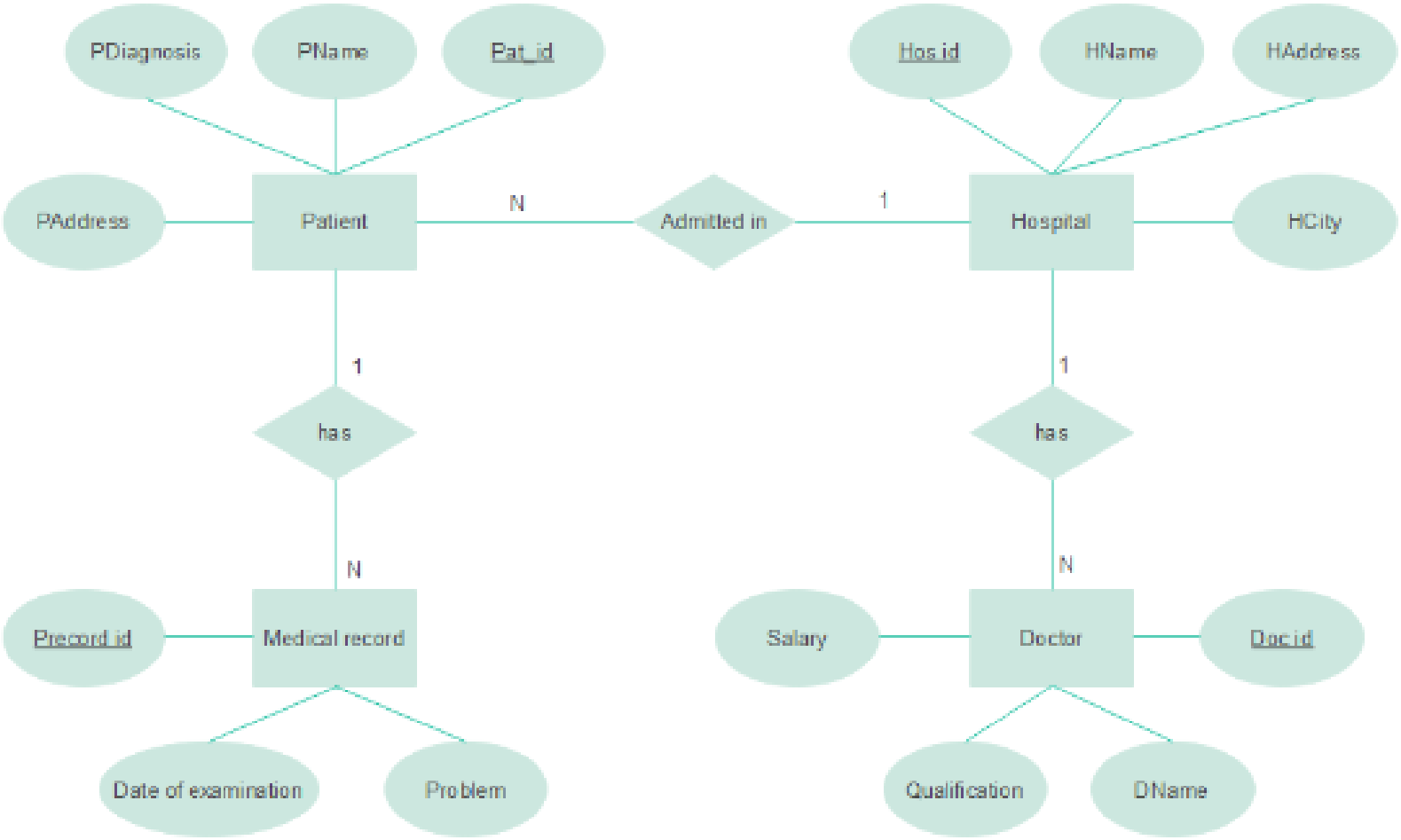
- Bus - Company owns busses and will hold information about them.
- Route - Buses travel on routes and will need described.
- Town - Buses pass through towns and need to know about them
- Driver - Company employs drivers, personnel will hold their data.
- Stage - Routes are made up of stages
- Garage - Garage houses buses, and need to know where they are.

©Silberschatz, Korth and Sudarshan

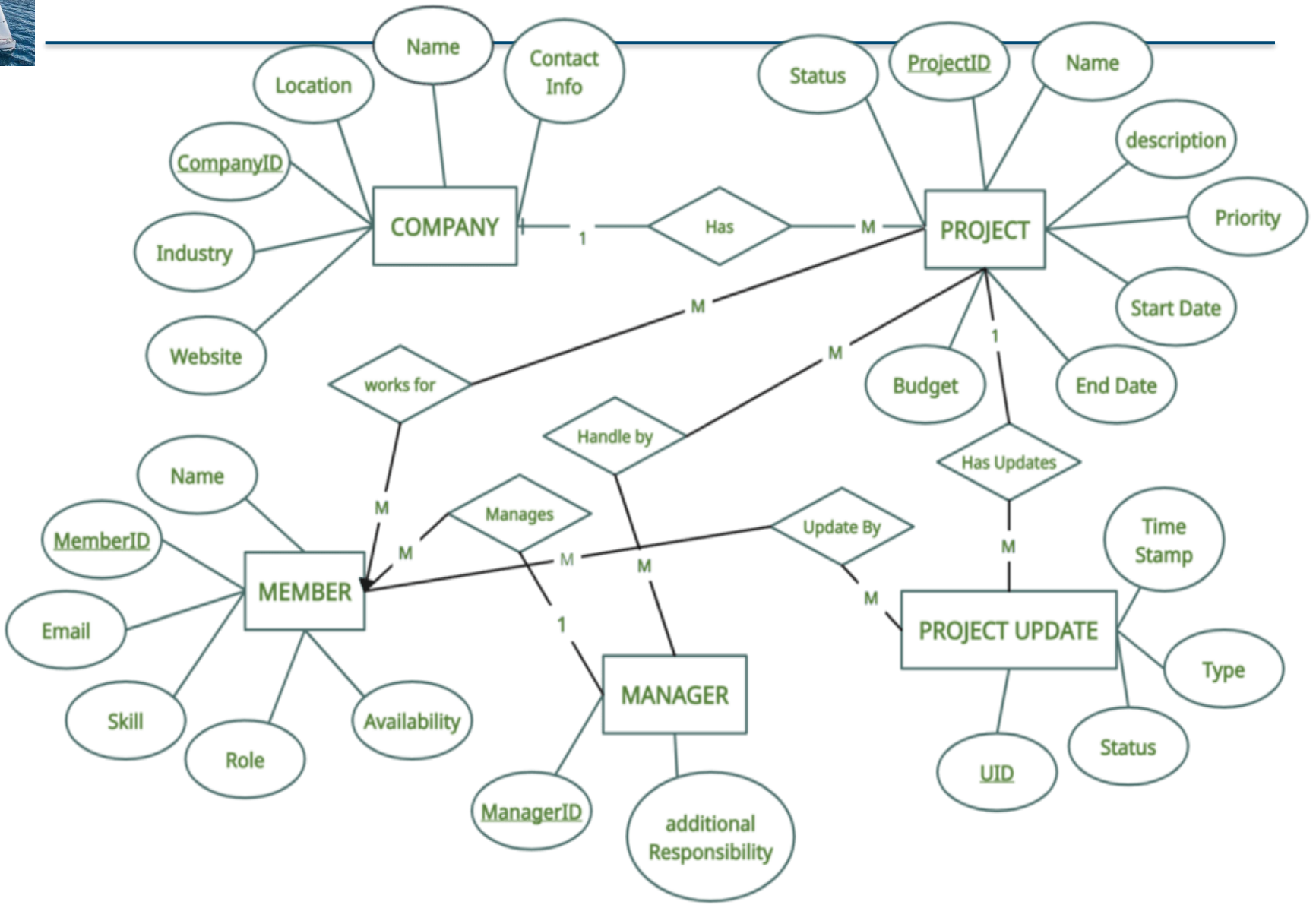

Name
Address
Specialization
Date
Type
Details
DOCTORS
Treats
PATIENTS
Report
Qualification
Disease
Treatmen
Log
Date
TESTS
Type
description



ER Diagrams of Hospital Management System






Name
Location
Contact Info
CompanyID
COMPANY
Industry
Website
1
Has
M
Status
ProjectID
Name
description
Priority
PROJECT
Start Date
End Date
Budget
1
M
M
works for
M
Handle by
Has Updates
Name
MemberID
Email
Skill
Role
Availability
MEMBER
M
M
Manages
1
M
Update By
M
M
Time Stamp
Type
Status
UID
PROJECT UPDATE
MANAGER
ManagerID
additional Responsibility



End of Chapter 6

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use